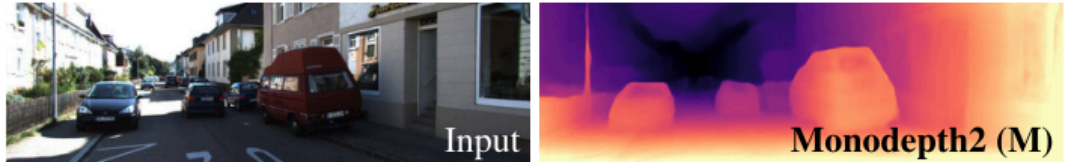
Self-Supervised Learning for Robot Perception

INF8422: Robot Perception and Spatial Intelligence

Prof. Pierre-Yves Lajoie · Polytechnique Montréal

POLYTECHNIQUE MONTRÉAL

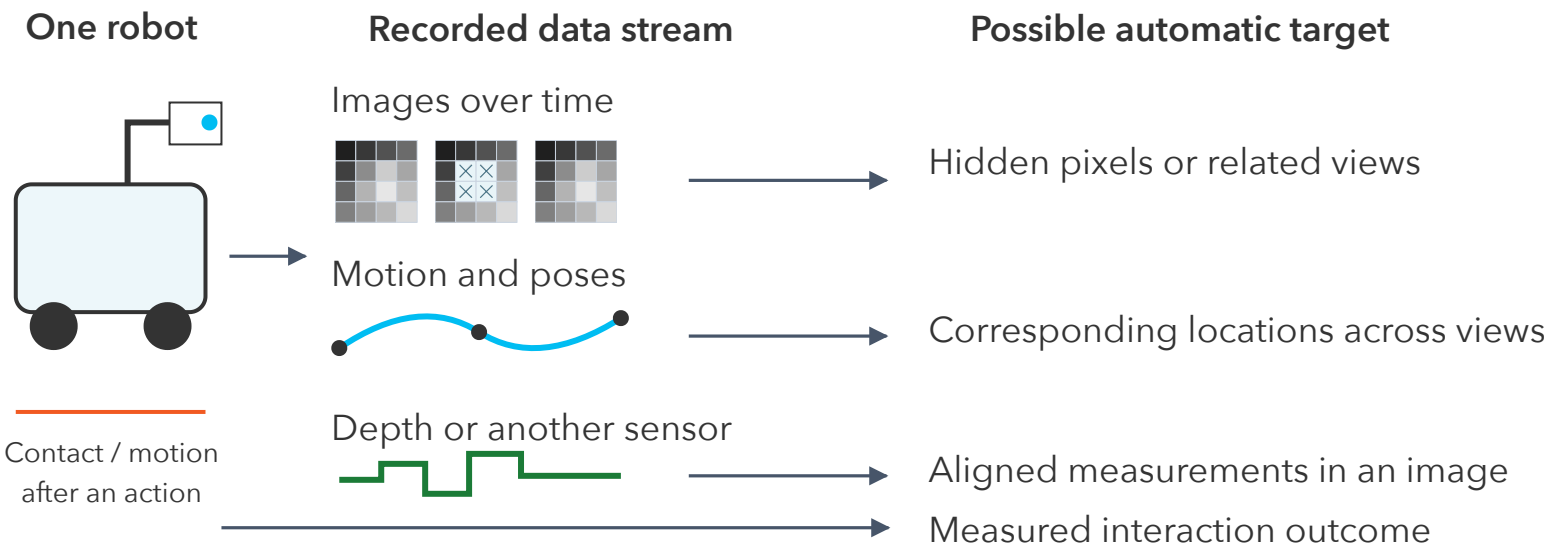


Camera observation and estimated depth
Godard et al., Monodepth2, ICCV 2019, Fig. 1 (top row)

SECTION 01

Self-supervised learning mechanics

Supervision available to a robot

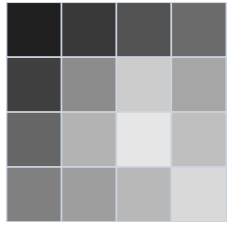


The target needs a construction rule

Recorded data can supply a training signal when we define a valid relationship between observations.

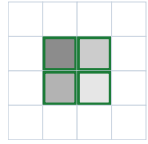
Definition of self-supervised learning

Self-supervised learning trains a model using supervisory signals derived automatically from the data itself, without requiring manually annotated labels for that learning stage.



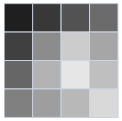
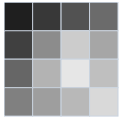
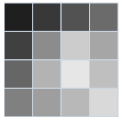
Same observation

→ Predict an object label ← **"object"** Human annotation

→ Predict hidden pixels ←  Copied from the observation

> The origin of the training target determines the supervision setting.

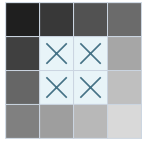
Supervised, self-supervised, and semi-supervised learning

Supervised		Manually labeled examples	→ Annotation-based loss
Self-supervised		Automatically constructed targets	→ Data-derived loss
Semi-supervised		Some labeled + unlabeled examples	→ Supervised + unlabeled-data terms

The loss (cost) function alone does not identify the setting

Squared error and cross-entropy can compare predictions to either manual or automatic targets.

Pretext tasks and automatic targets



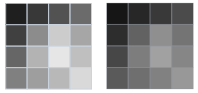
Hidden-pixel prediction

Input: visible information

→

Withheld observed pixels

Target: selected from the original image



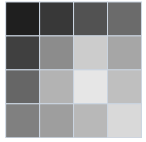
Related-view matching

Input: two images with a known relationship

→

A known pair association

Target: known relation (e.g., rotation, translation, overlap)



Direct outcome prediction

Input: an observation before the action

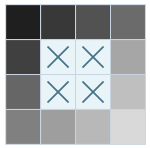
→

A measured traversal cost: 0.4

Target: measurements after the action

A pretext task builds a representation through prediction or matching.

Pretraining and optional fine-tuning



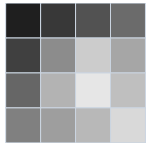
Encoder fθ

Learn a representation

Pretext prediction

Target from the data

Reuse the learned encoder



Encoder fθ

Frozen or adapted

Task head hβ

Object category, for example

Manual task labels can enter here for adaptation or evaluation.

Two adaptation choices

Keep the encoder fixed and train a task head, or fine-tune the encoder together with the head.

Direct learning from measured outcomes

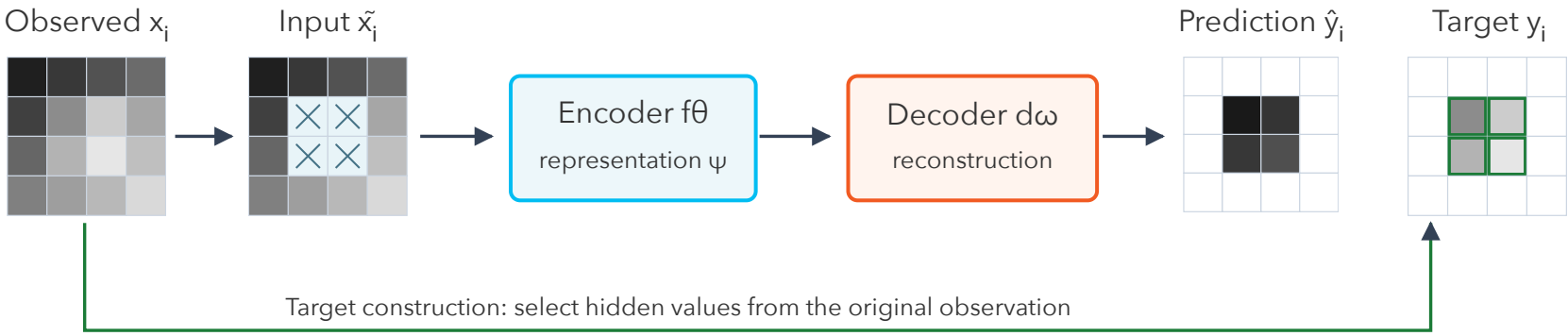
See the terrain **Execute a known fixed motion** **Measure the outcome**



Automatic target Illustrative cost = 1 − measured / commanded = **0.4**

Earlier camera image → **Encoder + task head** → Predicted traversal cost

Masking: Input, prediction, and target

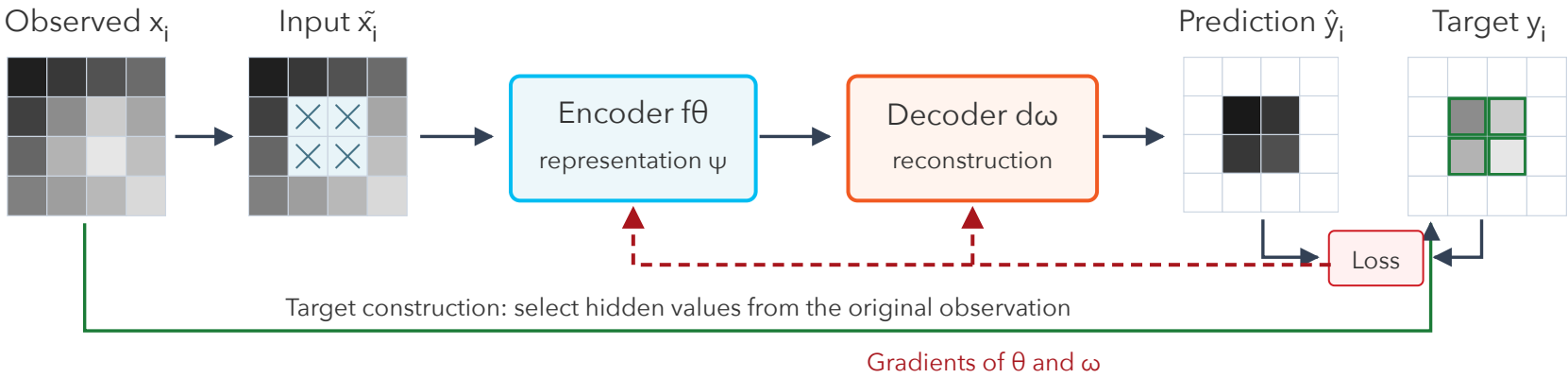


$$y_i = (x_{i,k})_{k \in \mathcal{M}}$$

Copy the withheld observed values into the target branch.

i identifies an observation · Θ contains the encoder and decoder parameters · M selects hidden locations

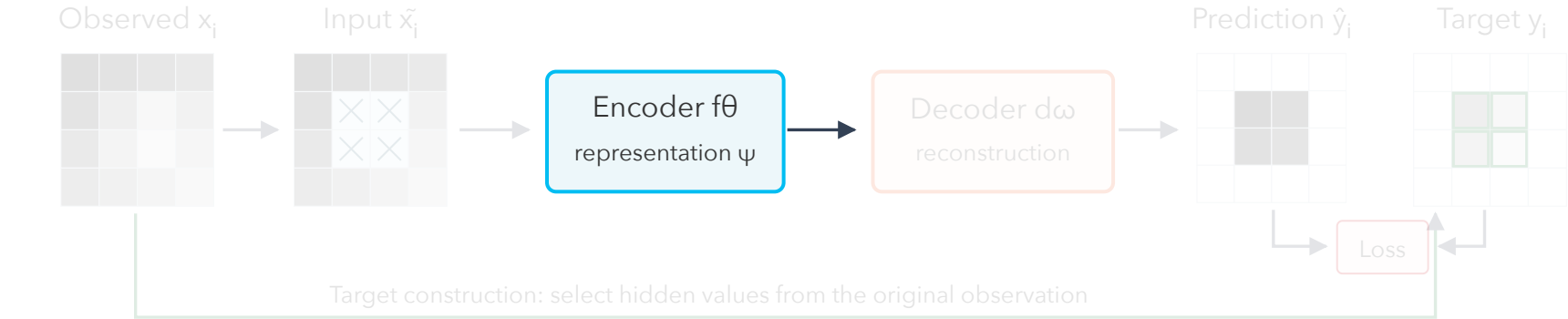
Masking: Loss and parameter update



$$\Theta_{n+1} = \Theta_n - \eta \nabla_\Theta \mathcal{L}$$

n counts optimization steps. η controls the size of each parameter update.

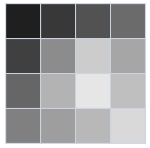
Masking: What remains after training



Keep fθ · replace dω with a task head when appropriate

$$\psi = f_\theta(x) \qquad \hat{y}^{\text{task}} = h_\beta(\psi)$$

Use the encoder on a new observation. The task determines the output head.



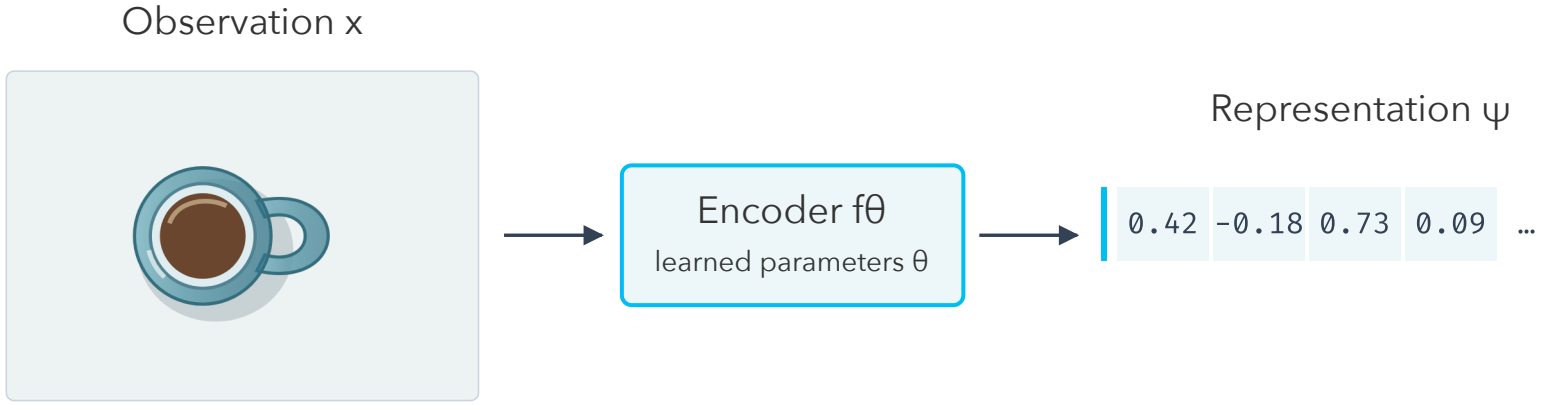
→ **Encoder fθ** → **Task head hβ** → Task output

> Evaluate the retained representation on the intended perception task.

SECTION 02

Representations for perception

Encoder representations



An internal description that a task can reuse

$$\psi = f_\theta(x)$$

A reusable description

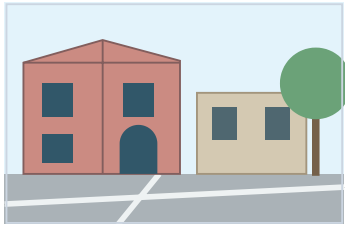
Representation quality depends on the information available for the intended task.

Global and dense features

Global features: one vector per image

Recognize a previously visited place

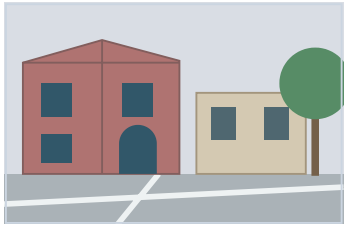
Query image



Encoder fθ ↓

`0.42` `-0.18` `0.73`

Stored image



Encoder fθ ↓

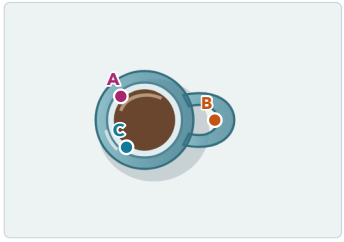
`0.40` `-0.20` `0.71`

Compare whole-image descriptors

Dense features: vectors at spatial locations

Find the corresponding handle point

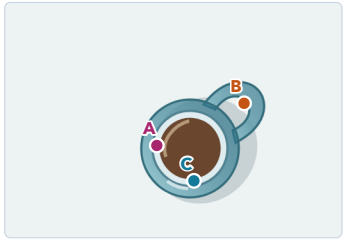
Reference image



Encoder fθ ↓



New image



Encoder fθ ↓



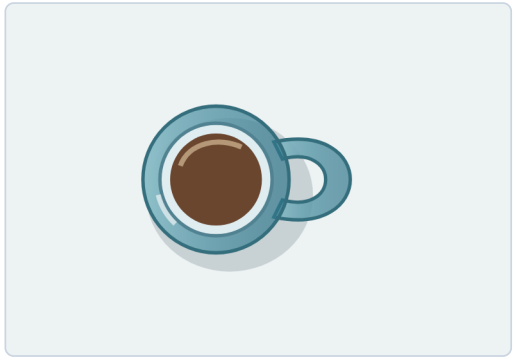
`-0.24` `0.81` `0.46`

Invariance to irrelevant changes

For **recognizing this mug**, lighting and background can be nuisance variables.

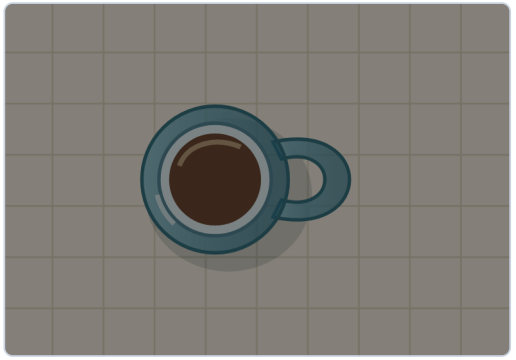
We therefore use **augmentations** during training.

Reference observation



change appearance

Changed observation



Desired identity output

same mug

Lighting: 0.55×

Background: pattern

Pixels change; the recognition decision stays stable.

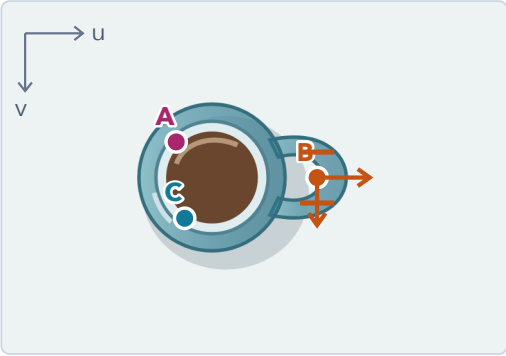
> Keep the information that identifies the object across the chosen changes.

Spatial information needed for action

For **grasping the handle**, its location and orientation must remain accessible.

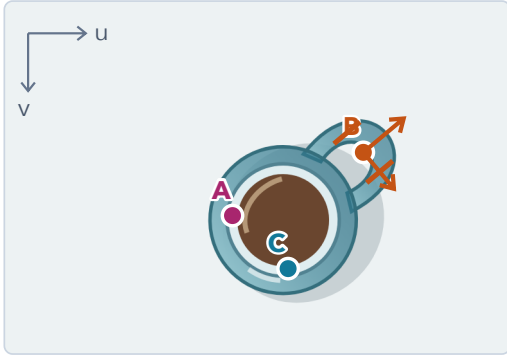
We therefore use **augmentations** during training.

Reference observation



move the mug

Changed observation



Grasp in image coordinates

(212.5, 90.1)

α = -40° · positive clockwise

B: same illustrative descriptor

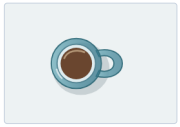
```
-0.24  0.81  0.46
```

The handle descriptor stays associated with B; its image location and grasp orientation change.

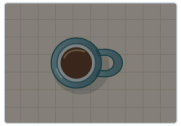
> Corresponding features can stay consistent while their spatial locations change.

Matching and prediction objectives

Match related observations



view 1 → fθ → ψ1



view 2 → fθ → ψ2

shared θ

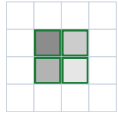
ψ1, ψ2 → Compare features

Known relation: same observation

Predict hidden information



input $\tilde{x}$ → fθ → ψ → dω → $\hat{y}$ → Loss ← Target y



Two mechanisms to follow

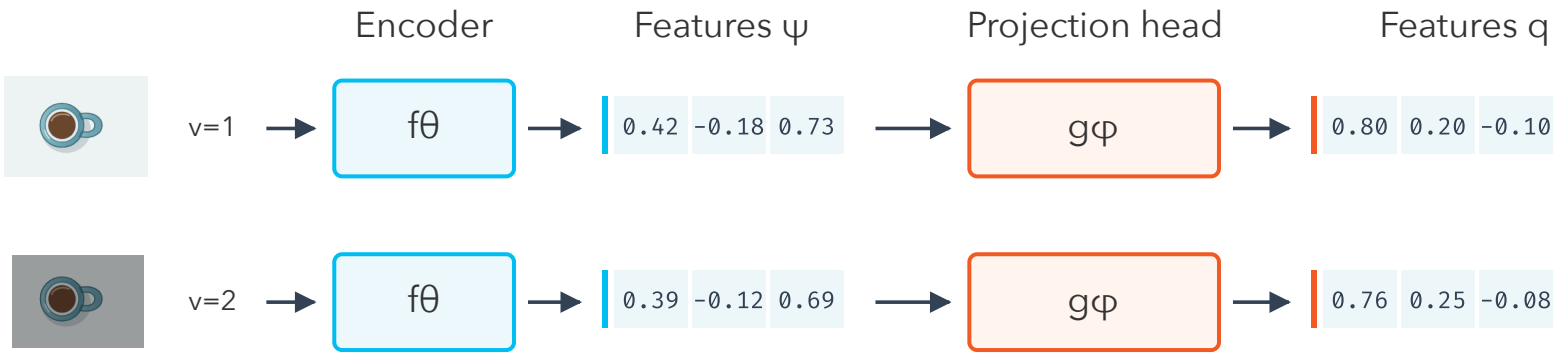
Matching uses a known relation between observations.

Masking uses observed information withheld from the input.

SECTION 03

Matching, collapse, and self-distillation

Encoder and projection head



Reuse the encoder representation

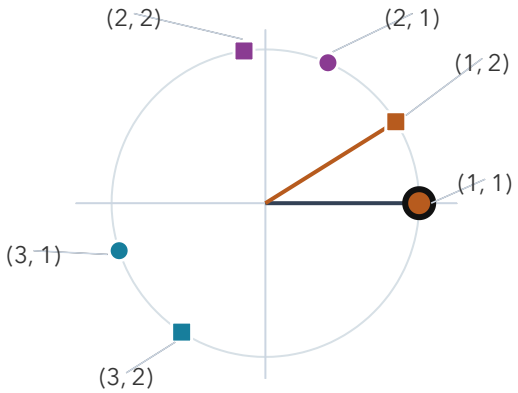
Compare projected features

$$\boldsymbol{\psi}_i^{(v)} = f_\theta(\tilde{\boldsymbol{x}}_i^{(v)}) \qquad \mathbf{q}_i^{(v)} = g_\phi(\boldsymbol{\psi}_i^{(v)})$$

The objective compares q. Many different downstream tasks can reuse encoder features ψ.

Similarity and temperature

Anchor (1, 1) · Positive (1, 2)



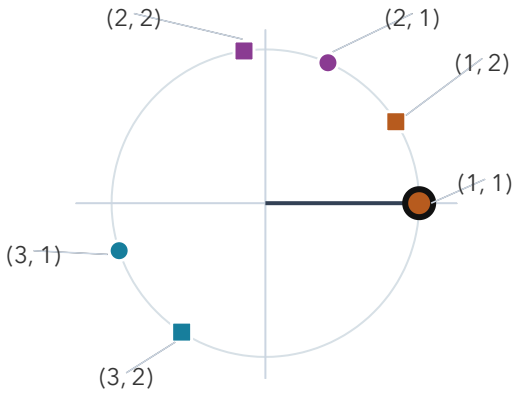
Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.848	60.9%
(2, 1) · negative	0.407	25.2%
(2, 2) · negative	-0.139	8.5%
(3, 1) · negative	-0.951	1.7%
(3, 2) · negative	-0.545	3.8%

τ = 0.50 · positive probability = 60.9% · ℓ = 0.496

$$\mathrm{sim}(\mathbf{q}_a, \mathbf{q}_b) = \frac{\mathbf{q}_a^\top \mathbf{q}_b}{\|\mathbf{q}_a\|_2 \|\mathbf{q}_b\|_2}$$

Contrastive loss

Anchor (1, 1) · Positive (1, 2)



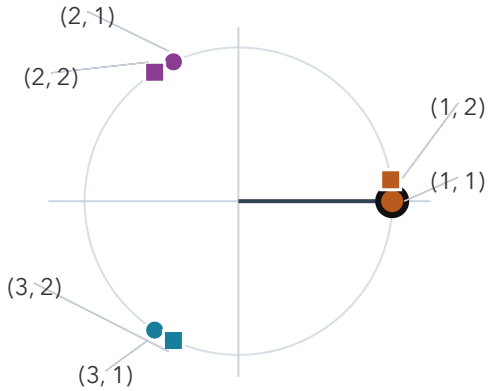
Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.848	60.9%
(2, 1) · negative	0.407	25.2%
(2, 2) · negative	-0.139	8.5%
(3, 1) · negative	-0.951	1.7%
(3, 2) · negative	-0.545	3.8%

τ = 0.50 · positive probability = 60.9% · ℓ = 0.496

$$\ell_i^{(1)} = -\log \frac{\exp(\mathrm{sim}(\mathbf{q}_i^{(1)}, \mathbf{q}_i^{(2)})/\tau)}{\sum_{(j,v)\neq(i,1)} \exp(\mathrm{sim}(\mathbf{q}_i^{(1)}, \mathbf{q}_j^{(v)})/\tau)}$$

How the objective changes representations

Augmented view → **fθ** → ψ → **gφ** → q

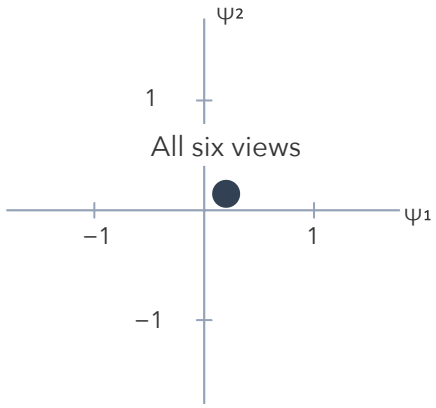


Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.990	82.6%
(2, 1) · negative	-0.423	4.9%
(2, 2) · negative	-0.545	3.8%
(3, 1) · negative	-0.545	3.8%
(3, 2) · negative	-0.423	4.9%

τ = 0.50 · positive probability = 82.6% · ℓ = 0.192 · mean over six anchors = 0.189

Increase agreement with the positive relative to the competing views.

Constant representation: Collapse



Three distinct pairs

Mean pair-agreement loss: **0.0000**

Feature spread: (0.000, 0.000)

One constant vector erases distinctions between observations.

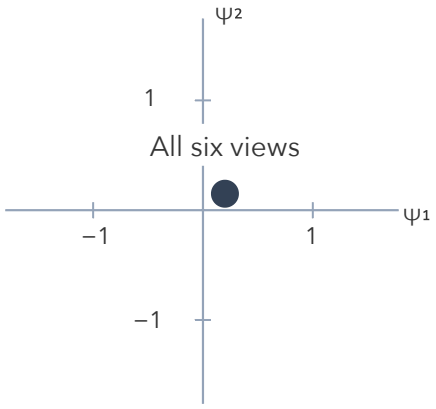
c = (0.20, 0.15)

$$f_\theta(x) = \mathbf{c} \text{ for every } x \quad \Longrightarrow \quad \ell_{\text{agree},i} = 0$$

The learning objective must avoid collapse of the representation as a possible optimal solution.

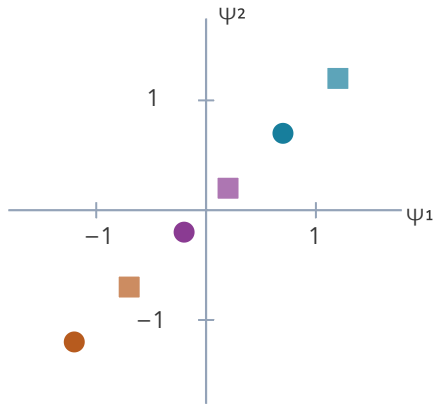
Loss of feature dimensions

Point: complete collapse



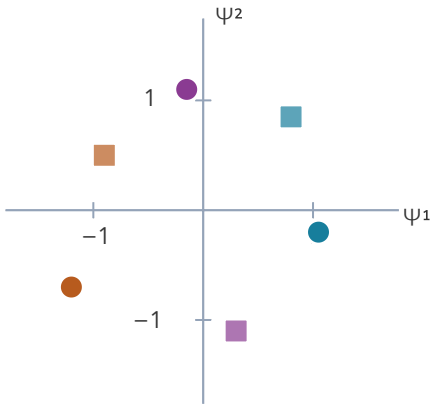
Std: (0.00, 0.00)

Line: one varying direction



Std: (0.89, 0.89)

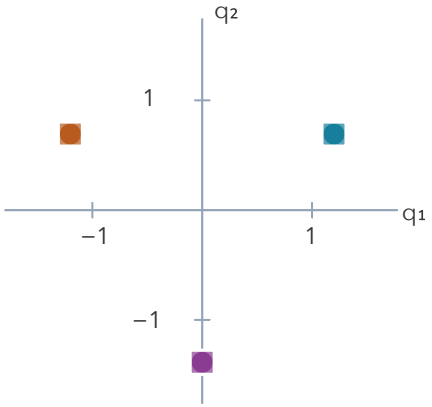
Two varying directions



Std: (0.91, 0.88)

Regularization: Variance-Invariance-Covariance (VICReg)

View → fθ → ψ → **gφ (expander)** → q · regularize q in both branches



Invariance · 0.0000

Distance between corresponding views

Variance · 0.0000

Per-dimension standard-deviation floor

Covariance · 0.0000

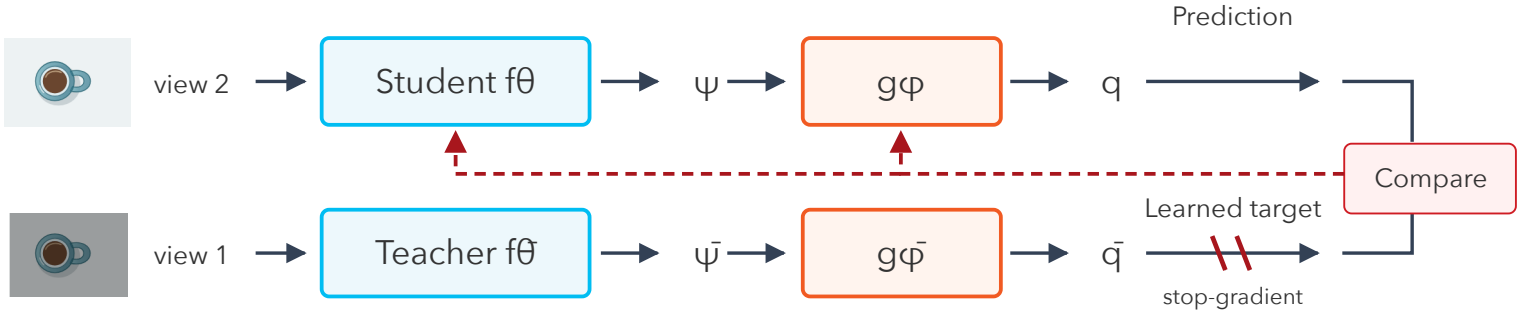
Squared off-diagonal covariance

$$\mathcal{L}_{\text{VICReg}} = \lambda_{\text{inv}}\mathcal{L}_{\text{inv}} + \lambda_{\text{var}}\mathcal{L}_{\text{var}} + \lambda_{\text{cov}}\mathcal{L}_{\text{cov}}$$

Covariance discourages redundant feature directions.

Bardes et al. · VICReg · ICLR 2022

Self-distillation: Learning targets from a teacher

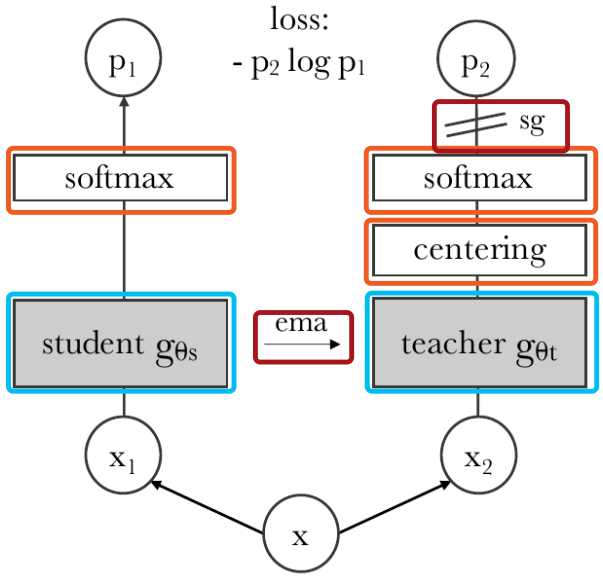


Dashed: gradients update θ and φ · no teacher gradient

A target that evolves during learning

The student predicts a teacher output from a different view of the observation.

DINO architecture



Caron et al. · DINO · ICCV 2021 · main-paper Fig. 2

Student and teacher include encoder + head

$$g_{\theta_s} \leftrightarrow g_\phi \circ f_\theta$$

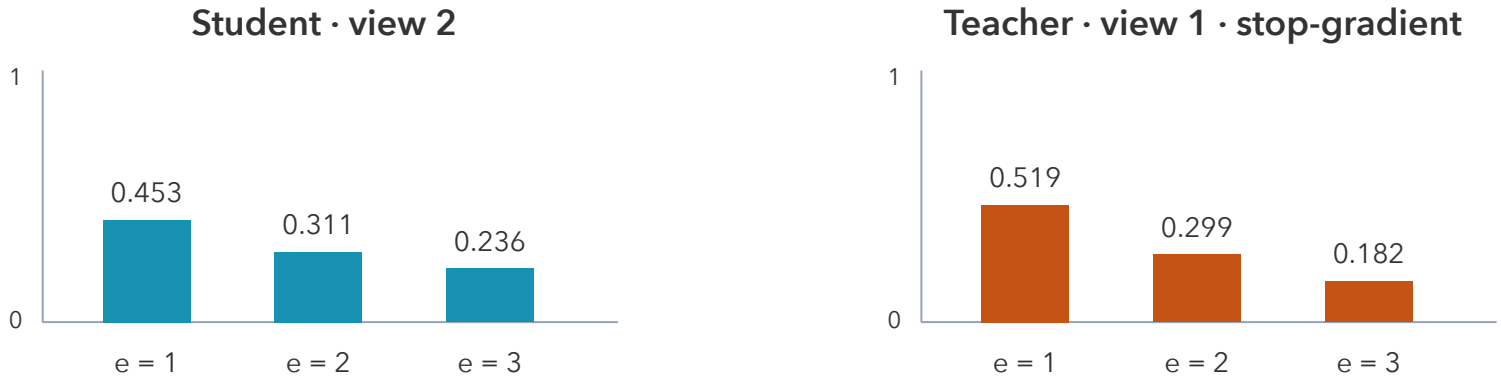
$$g_{\theta_t} \leftrightarrow g_{\bar{\phi}} \circ f_{\bar{\theta}}$$

Center the teacher outputs, then apply separate softmax temperatures.

Stop the teacher gradient; update its parameters by a moving average.

Match teacher and student distributions; update their parameters by different rules.

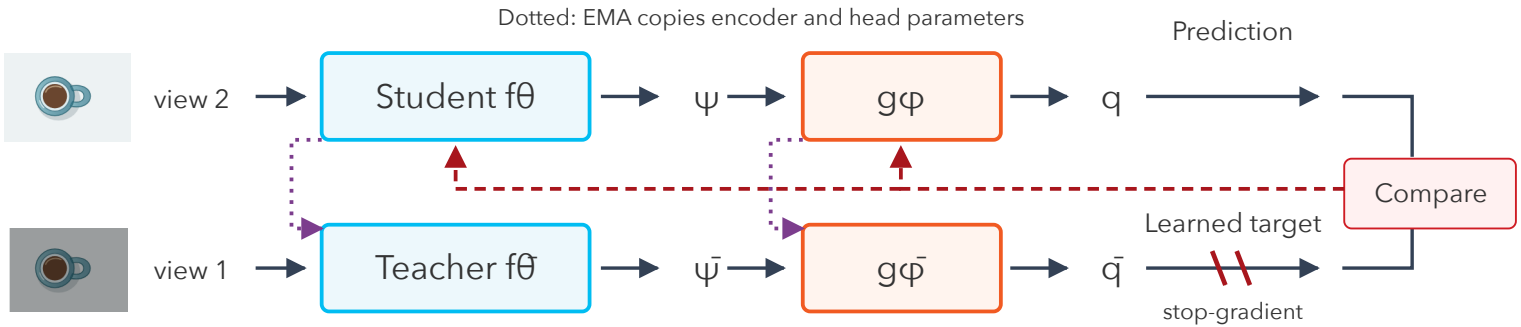
Distribution matching in DINO



$$\ell_{\text{DINO}} = -\sum_{e} \rho_{\text{teach},e} \log \rho_{\text{stu},e}$$

Computed one-view-pair cross-entropy: **1.0230**

Gradient updates and moving averages



Dashed: gradients update θ and φ · no teacher gradient

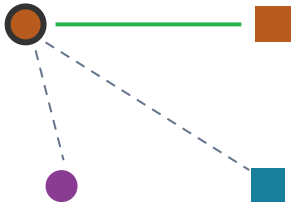
$$\bar{\theta}_{n+1} = \mu\bar{\theta}_n + (1-\mu)\theta_{n+1}$$

Encoder: θ = 0.488 · θ̄ = 0.578 Head: φ = 0.830 · φ̄ = 0.966

After a batch: update the teacher head φ̄ using a moving average.

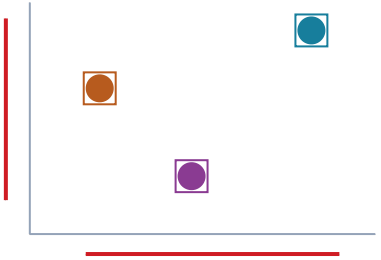
Summary of representation learning

SimCLR



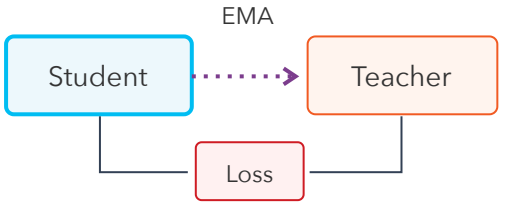
Competing views

VICReg



Spread + decorrelation

DINO



Centered, sharpened target

Teacher distribution

Observation → encoder → features → constructed relation or target → loss → parameter update

The goal of a good representation

Keep the information useful for the intended task.

Ignore the rest.

SECTION 04

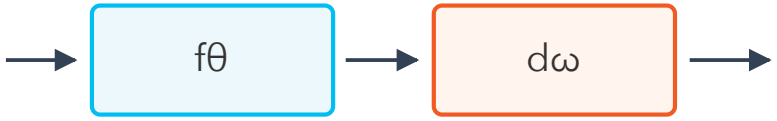
Reconstruction and feature prediction

Reconstruction as a training signal

Partially observed input







Predicted pixels

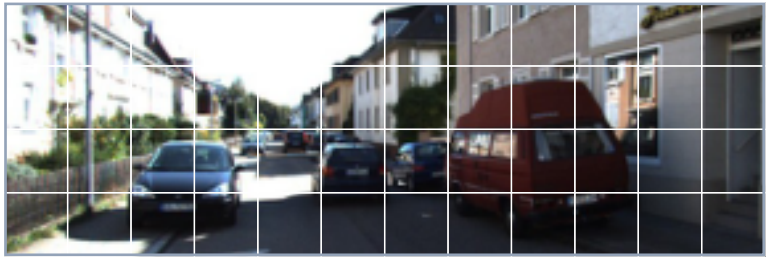
Original observed pixels supply the target.

Withhold information from the prediction input; keep it for the target.

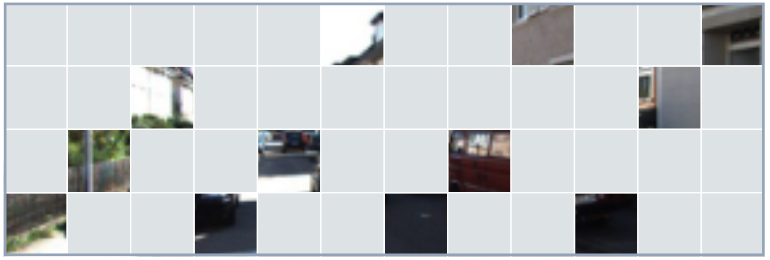
> The original observation supplies a target without a human annotation.

Visible and masked patches

Original observation x



Visible patches V · hidden patches M

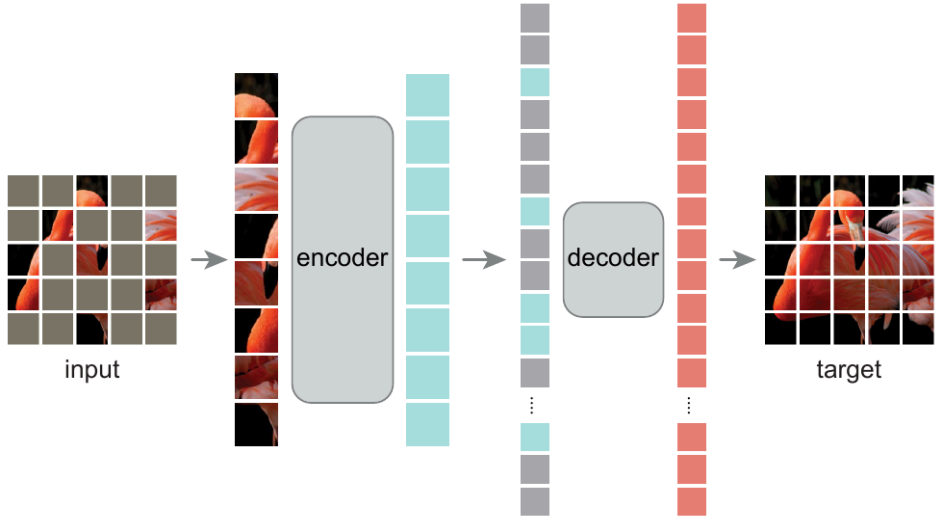




Visible patch tokens + their positions

The MAE (masked autoencoder) encoder receives visible tokens, with their original positions.

MAE architecture

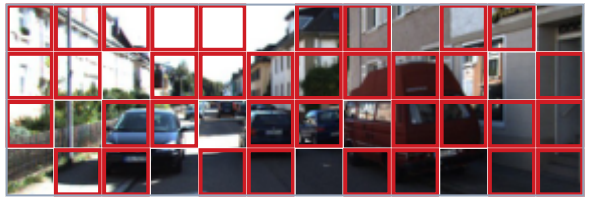


Mask tokens enter the decoder after visible-patch encoding.

He et al., MAE, CVPR 2022 · Fig. 1 · arXiv v3

Reconstruction loss on masked locations

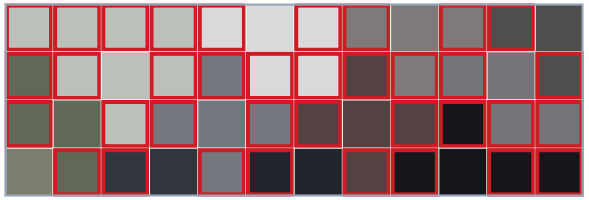
Original target pixels



Visible encoder input



Pixel prediction



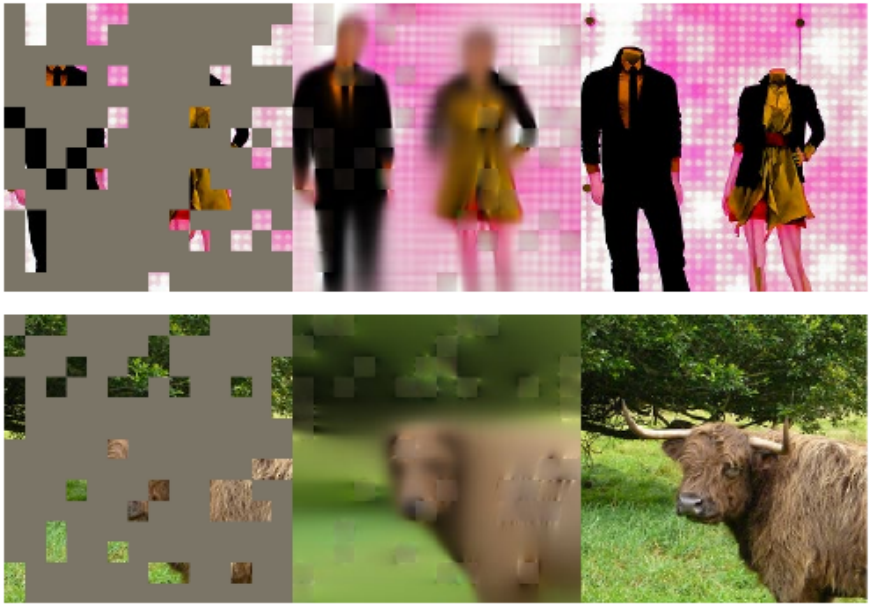
$$\mathcal{L}_{\text{pixel}} = \frac{1}{|\mathcal{M}|} \sum_{k \in \mathcal{M}} \|\hat{x}_k - x_k\|_2^2$$

> Average over 36 masked patches: Lpixel = 148.32

The basic loss acts on masked patches.

Plausible reconstruction and ambiguity

Masked input MAE reconstruction Original image



Multiple possible completions

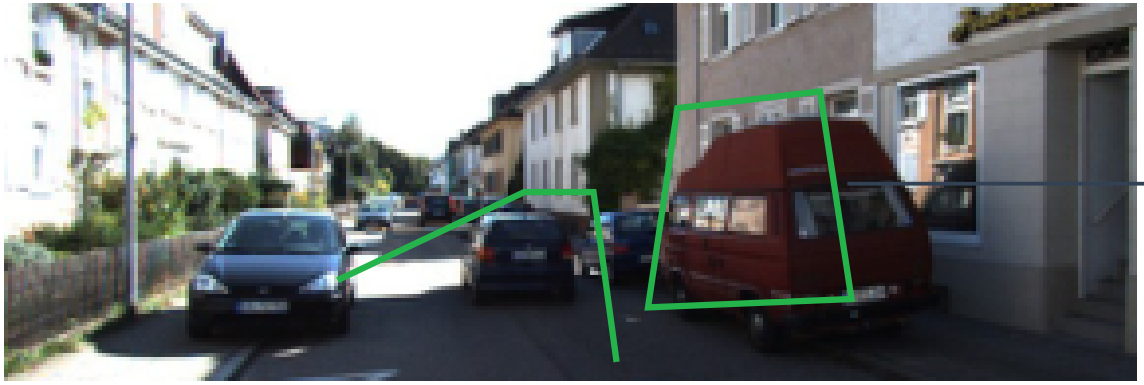
Visible context does not determine every hidden detail.

The outputs differ from the originals while retaining plausible scene structure.

Visual plausibility alone does not establish semantic understanding or robot-task accuracy.

He et al., MAE, CVPR 2022 · Fig. 3, two right-most examples · arXiv v3 · COCO validation images; model pretrained on ImageNet

What does a prediction target preserve?



Vehicle structure

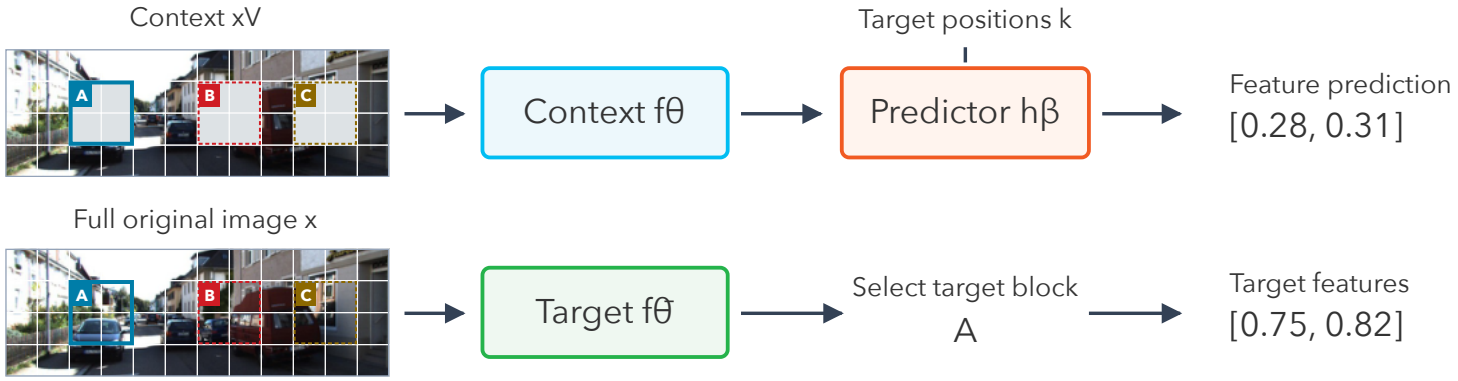
Potential task evidence

A robot task may place more value on object and road structure.

A target defines what prediction errors the model must reduce.

> The features worth learning depend on the task.

JEPA: Predicting in feature space

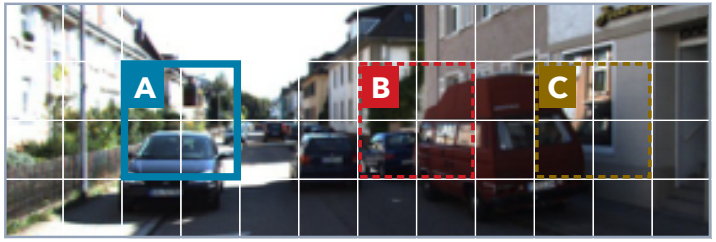


$$\psi_k^{\text{target}} = f_{\bar{\theta}}(x)_k$$

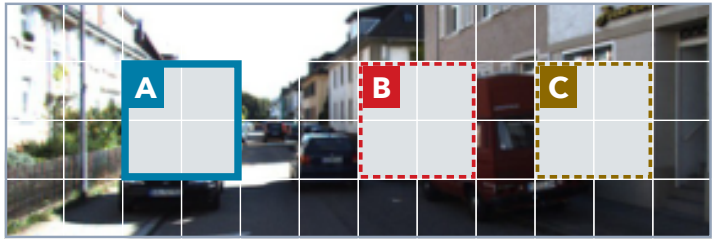
Compute the target in the intermediate feature space. No decoding into the output space (e.g., an image) is required. This supports learning more abstract concepts and more efficient training at scale.

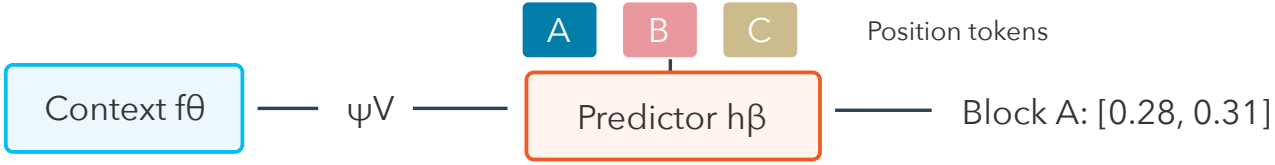
JEPA: Context and target positions

Original · spatial target blocks A, B, C



Context V · every target region removed

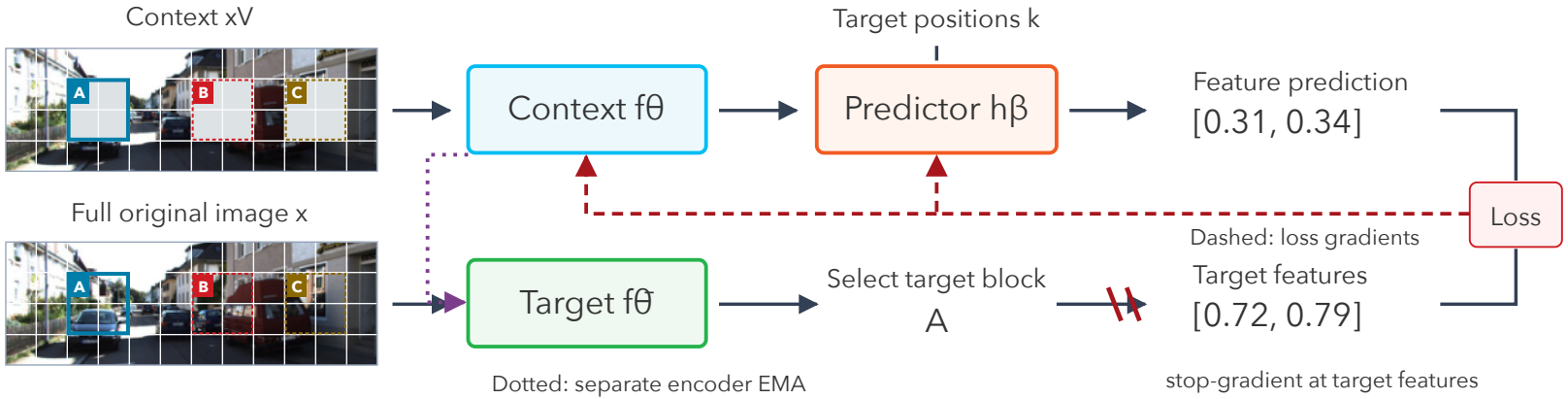




A position token identifies where to predict; it contains no hidden pixels.

$$\widehat{\psi}_k = h_\beta(f_\theta(x_V), k)$$

JEPA: Feature loss and target updates



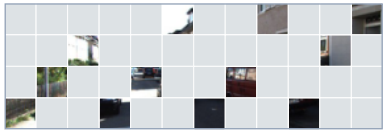
$$\bar{\theta}_{n+1} = \mu\bar{\theta}_n + (1-\mu)\theta_{n+1}$$

Lfeature = 0.1515 θ = 0.833 · β = 0.738 · θ̄ = 0.967

EMA transfers context-encoder parameters.

Pixel targets and feature targets

MAE · pixel target



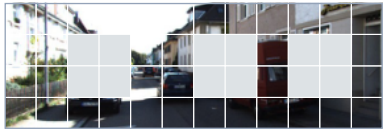
fθ → dω → $\hat{x}_k$ ↔



Mask + position tokens

Observed pixels x_k

I-JEPA · feature target



fθ → hβ → $\hat{\psi}_k$ ↔ $sg(f_{\bar{\theta}}(x)_k)$

Target-position tokens

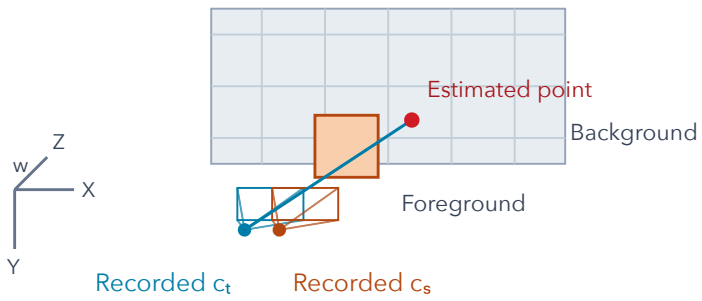
Targets across all image features

SECTION 05

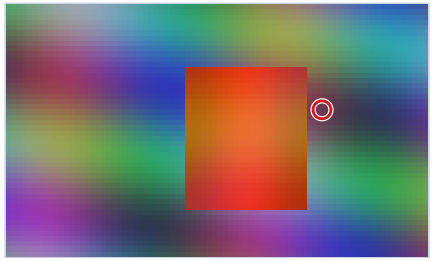
Geometric supervision

Learning depth from multiple observations

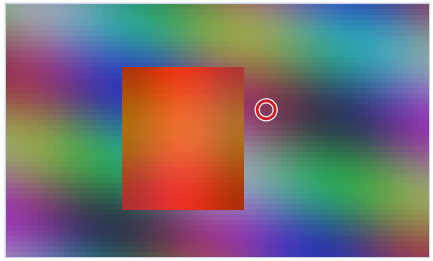
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

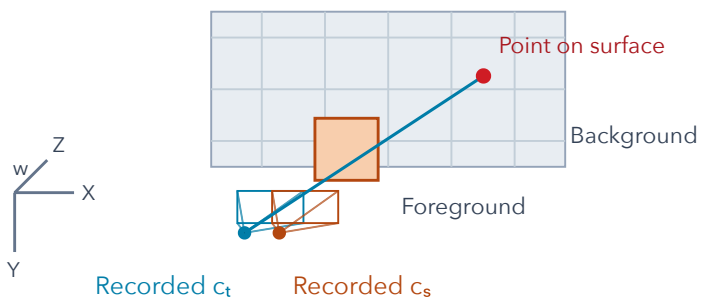
$\hat{D} = 3.50 \cdot \hat{p}$ in c_t: [1.06, -0.18, 3.50] · u_s = [49.31, 20.00]

Recorded image values provide the target; depth and relative pose are predicted.

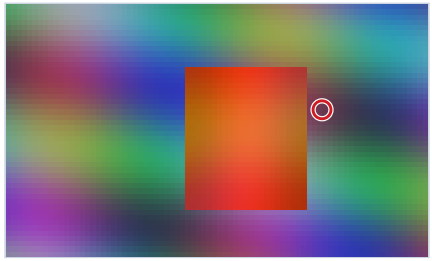
Scene structure and camera motion can turn recorded images into a learning signal.

Camera projection and intrinsics

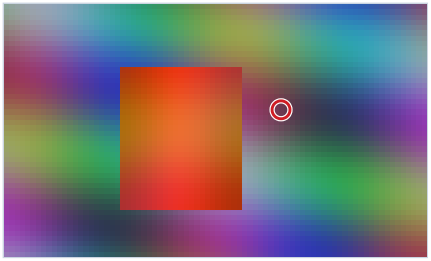
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

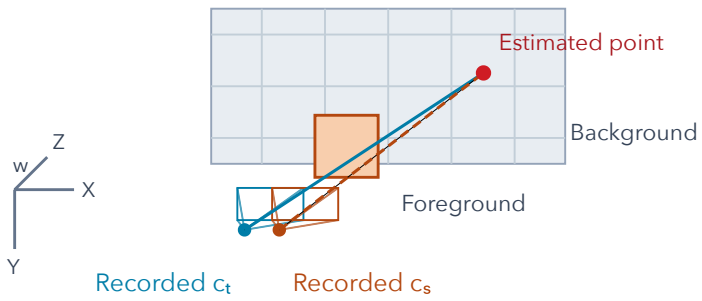
$$K_t = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

fx = fy = 68 · cx = 39.5 · cy = 23.5 · X right, Y down, Z forward

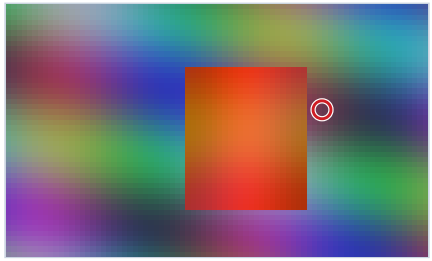
Optical-axis Z is depth; distance along a unit ray is a different quantity.

Back-projecting a target pixel

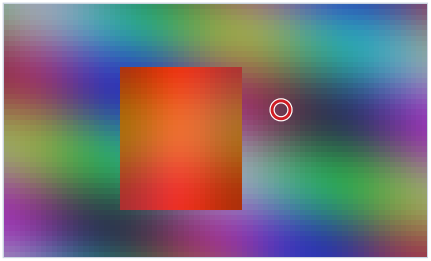
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

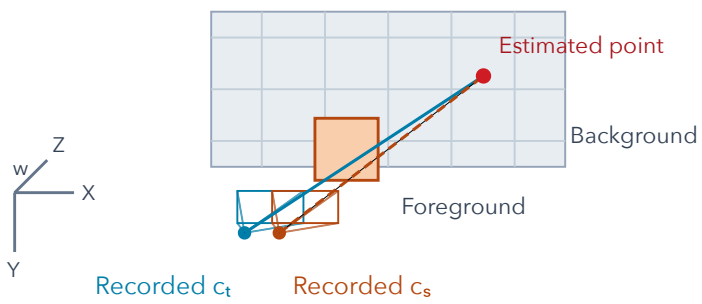
$$\hat{p}^{c_t} = \widehat{D}_t(\mathbf{u}) K_t^{-1} \tilde{\mathbf{u}}$$

$\hat{D}$ = 5.00 · $\hat{p}$ in c_t: [1.51, -0.26, 5.00] · u_s = [52.52, 20.00]

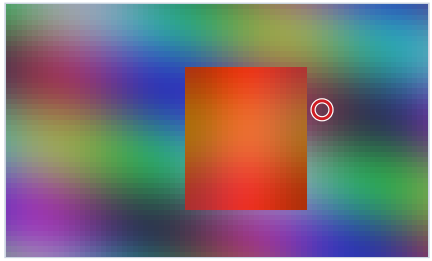
Changing estimated depth moves the hypothesis along a fixed target ray.

Effect of camera pose

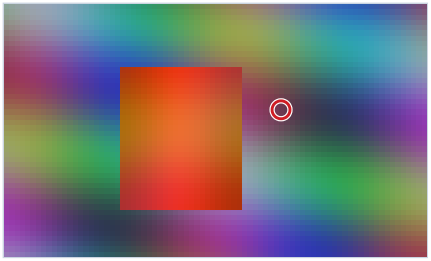
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

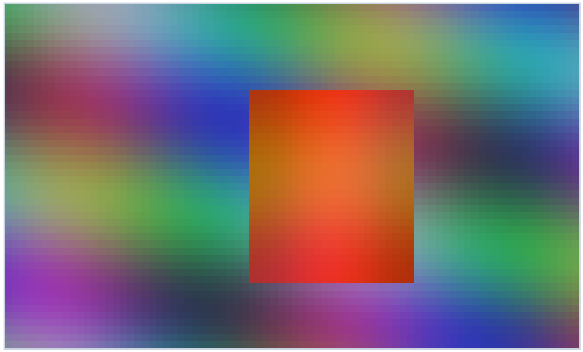
$$\hat{p}^{c_s} = \hat{R}^{c_s}_{c_t}\hat{p}^{c_t} + \hat{t}^{c_s}_{c_t}$$

$\hat{p}$ in c_t: [1.51, -0.26, 5.00] · $\hat{p}$ in c_s: [0.96, -0.26, 5.00]

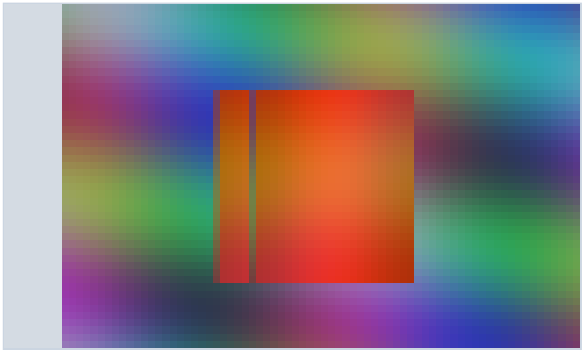
Changing the pose estimate changes coordinates and sampling; recorded images stay fixed.

Photometric reconstruction loss

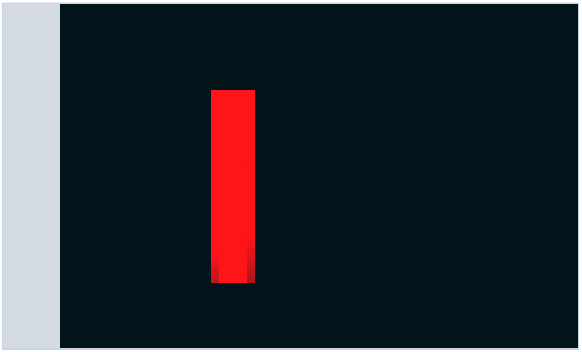
Recorded target I_t



Reconstructed target $\hat{I}_t$



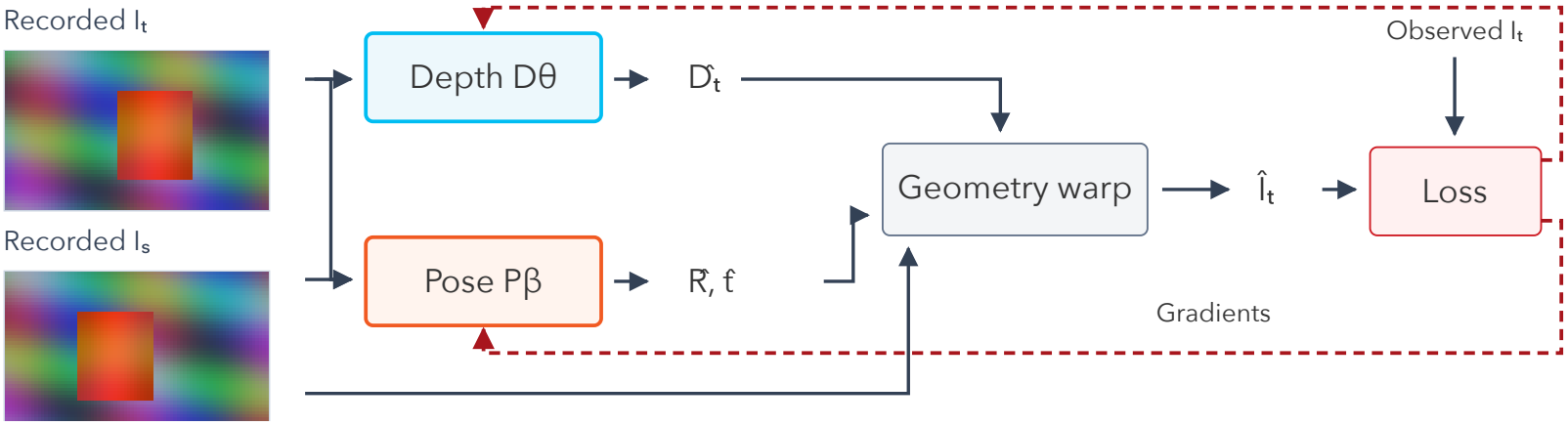
RGB L1 error · brighter = larger



$$\mathcal{L}_{\text{photo}} = \frac{1}{|\Omega|} \sum_{\mathbf{u} \in \Omega} \| I_t(\mathbf{u}) - \hat{I}_t(\mathbf{u}) \|_1$$

Mean RGB L1 = 0.0448 · 3456 / 3840 pixels in Ω

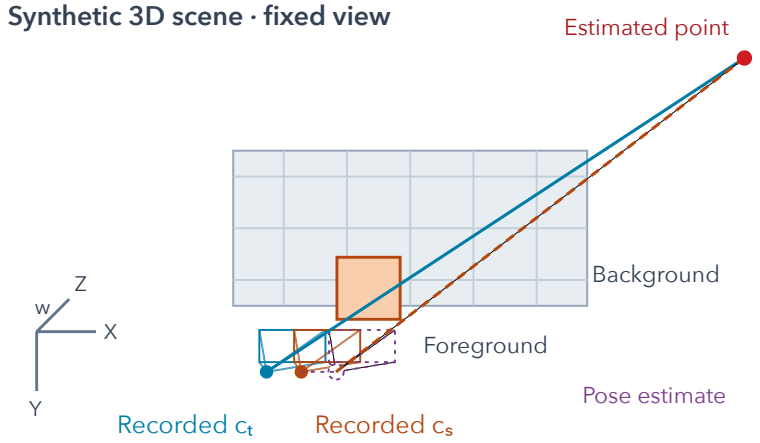
Updating depth and pose networks

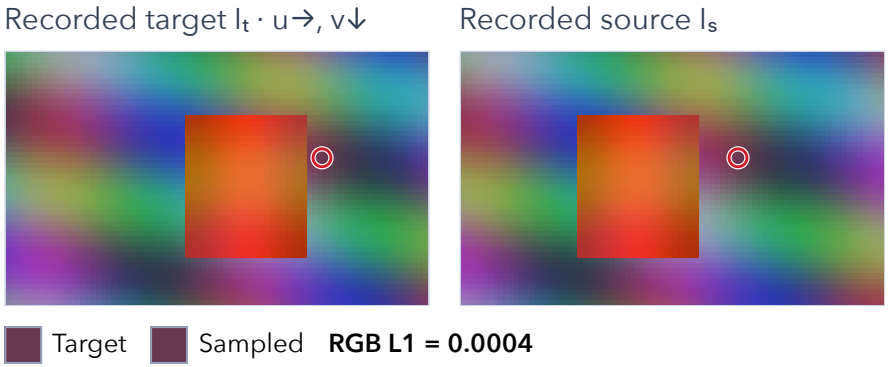


$$\Theta_{n+1} = \Theta_n - \eta \nabla_\Theta \mathcal{L}_{\text{photo}}, \qquad \Theta = (\theta, \beta)$$

Computed scalar-model update: lower pixel error does not determine metric scale.

Monocular scale ambiguity





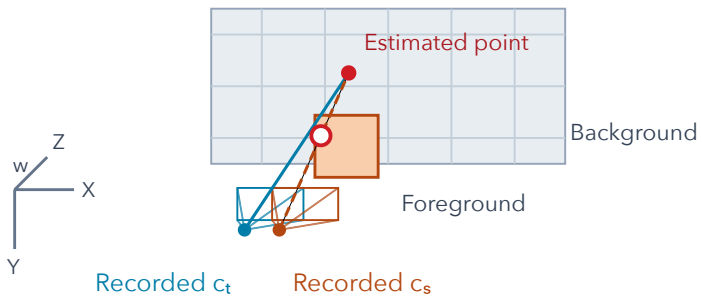
$$\pi(K_s\,\kappa\hat{p}^{c_s}) = \pi(K_s\hat{p}^{c_s}), \qquad \kappa > 0$$

$\hat{D}$ = 10.00 · $\hat{p}$ in c_t: [3.01, -0.51, 10.00] · u_s = [52.52, 20.00]

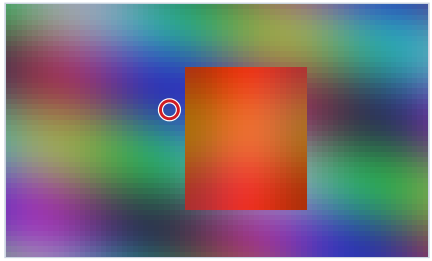
Scale the depth and every relative-translation component together; pixels stay fixed.

Occlusion limitations

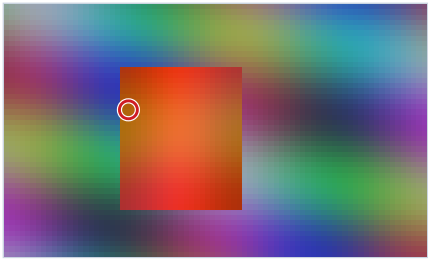
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Target Sampled **RGB L1 = 1.2332**

Source marker follows the computed projection.

$Z_s = 5.00 > 0$ · $u_s = [23.52, 20.00]$ · source ray hits the foreground first

A positive, in-bounds projection can still hit an occluded surface.

Texture and reflectance failures



Godard et al., Monodepth2, ICCV 2019 · Fig. 8 · arXiv v4

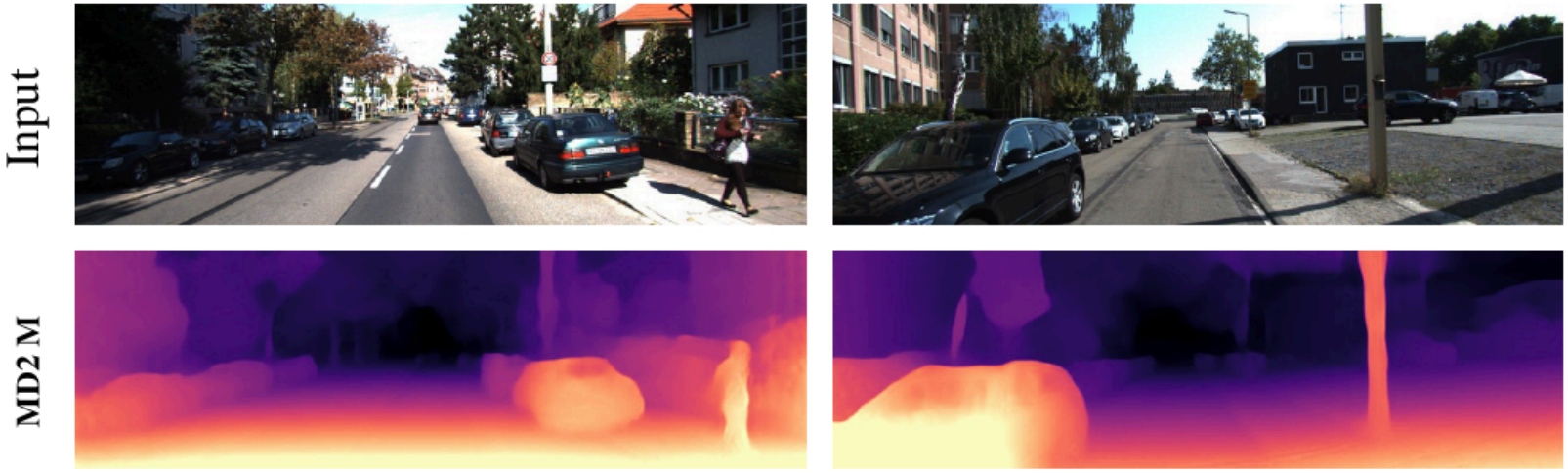
Appearance consistency can fail

Top: distorted, reflective, and color-saturated regions.

Bottom: ambiguous boundaries and intricate shapes can also be difficult.

These are the paper's examples, with its original method labels and highlighted regions.

Learned depth on real images



Recorded camera images above; Monodepth2 monocular depth predictions below.

> Metric depth requires additional supervised fine-tuning against 3D data (e.g., LiDAR).

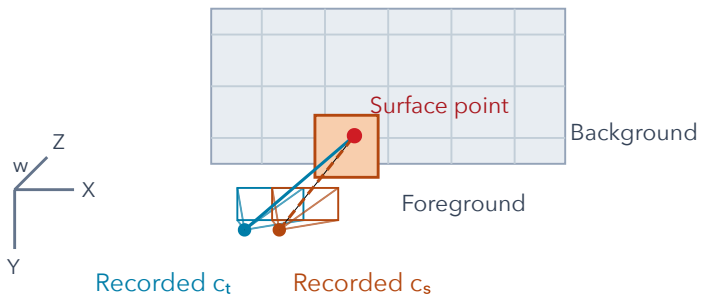
Godard et al., Monodepth2, ICCV 2019 · Fig. 7, first two examples, Input and MD2 M rows · KITTI Eigen split · arXiv v4

SECTION 06

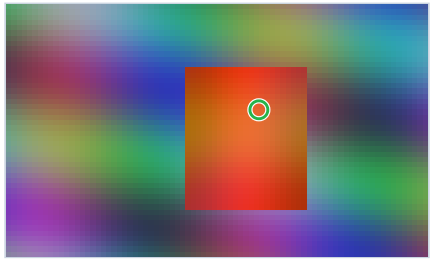
Dense correspondence

Geometrically generated pixel matches

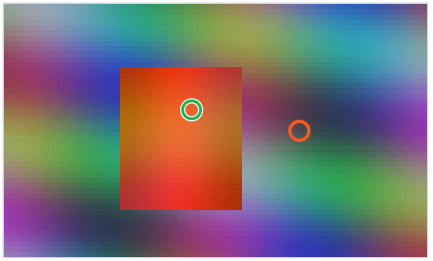
Synthetic 3D scene · fixed view



Target I_t



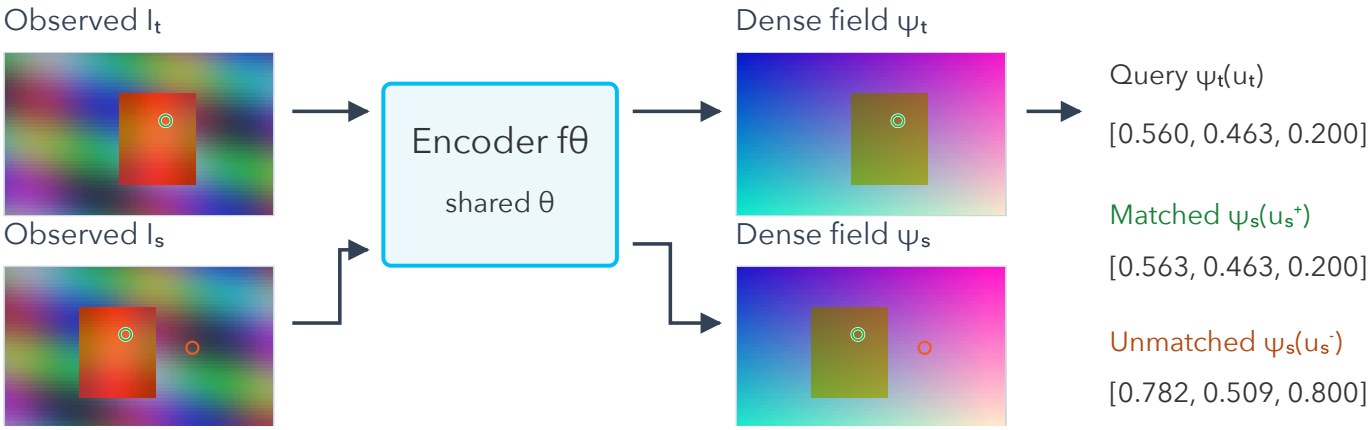
Source I_s



Green: same surface point · orange: different surface point

$$p^{c_t} = D_t(\mathbf{u}_t) K_t^{-1} \widetilde{\mathbf{u}}_t, \qquad \mathbf{u}_s = \pi(K_s(R_{c_t}^{c_s} p^{c_t} + t_{c_t}^{c_s}))$$

Learning dense descriptors



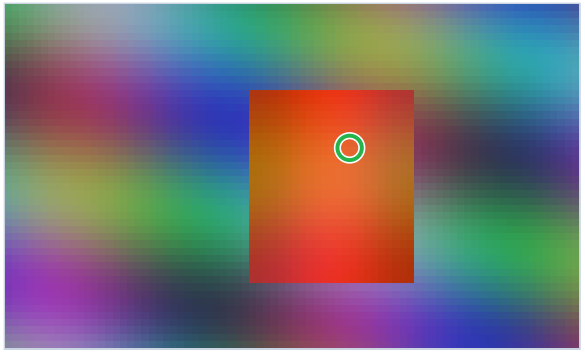
$$d_+ = \|\boldsymbol{\psi}_t(\mathbf{u}_t) - \boldsymbol{\psi}_s(\mathbf{u}_s^+)\|_2, \quad d_- = \|\boldsymbol{\psi}_t(\mathbf{u}_t) - \boldsymbol{\psi}_s(\mathbf{u}_s^-)\|_2$$

Matched distance $d_+ = 0.0033$ Unmatched distance $d_- = 0.6414$

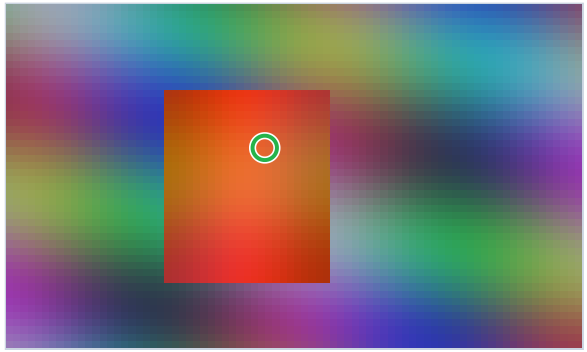
Nonmatching pairs provide a separation constraint.

Descriptor matching (searching for matches)

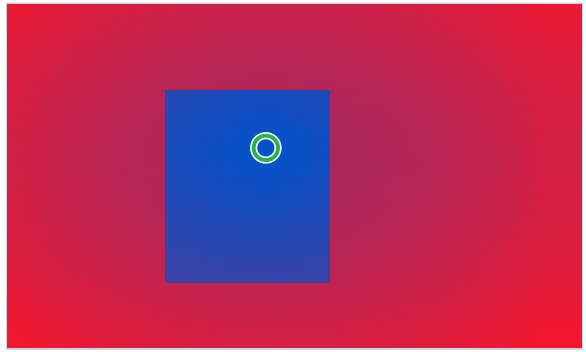
Query image I_t



Search image I_s



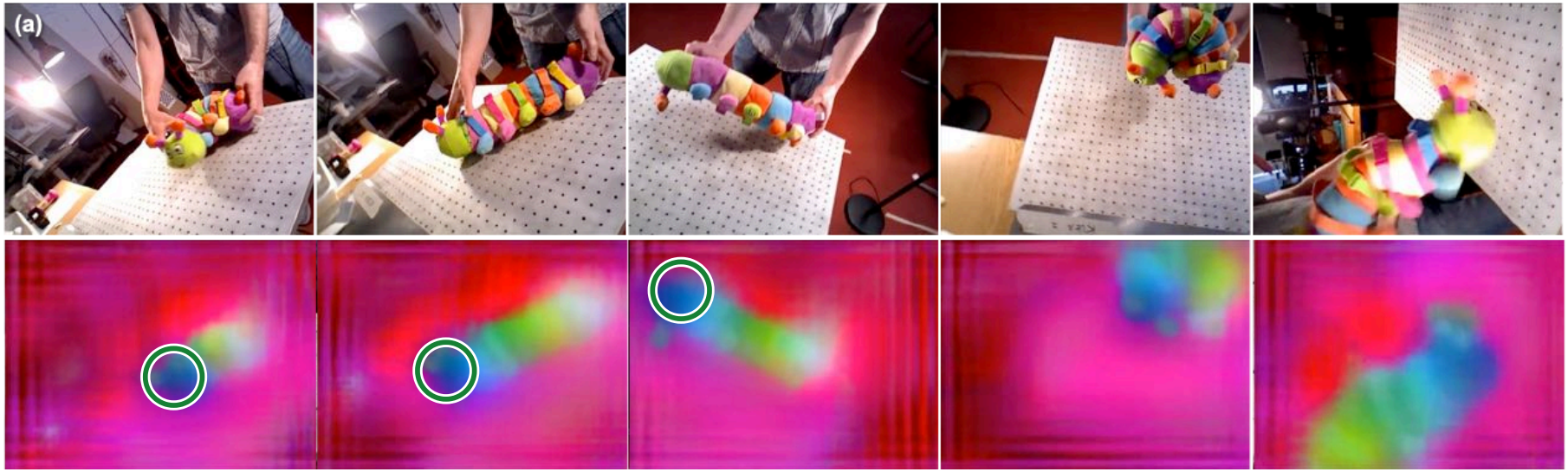
Distance · blue is closer



$$\mathbf{u}_s^* = \arg\min_{\mathbf{u}_s} \|\boldsymbol{\psi}_t(\mathbf{u}_t) - \boldsymbol{\psi}_s(\mathbf{u}_s)\|_2$$

Descriptor consistency across configurations

RGB observations above · dense descriptor colors below

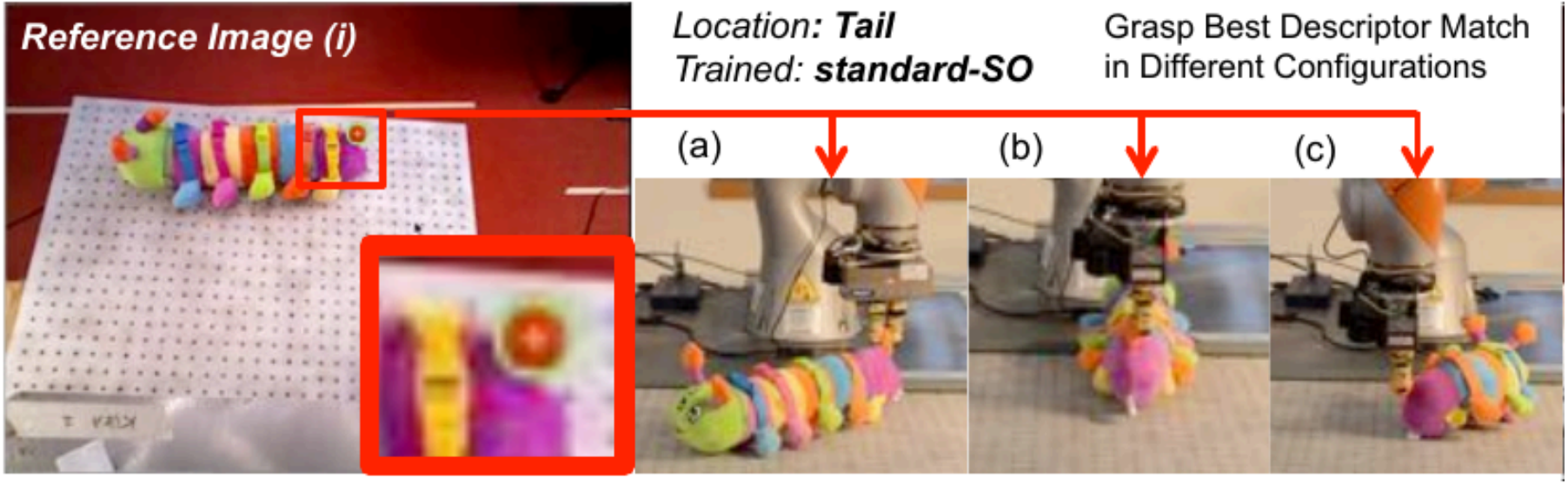


The same object changes configuration while corresponding regions retain similar feature colors.

> Feature colors encode descriptors, not manually assigned semantic part labels.

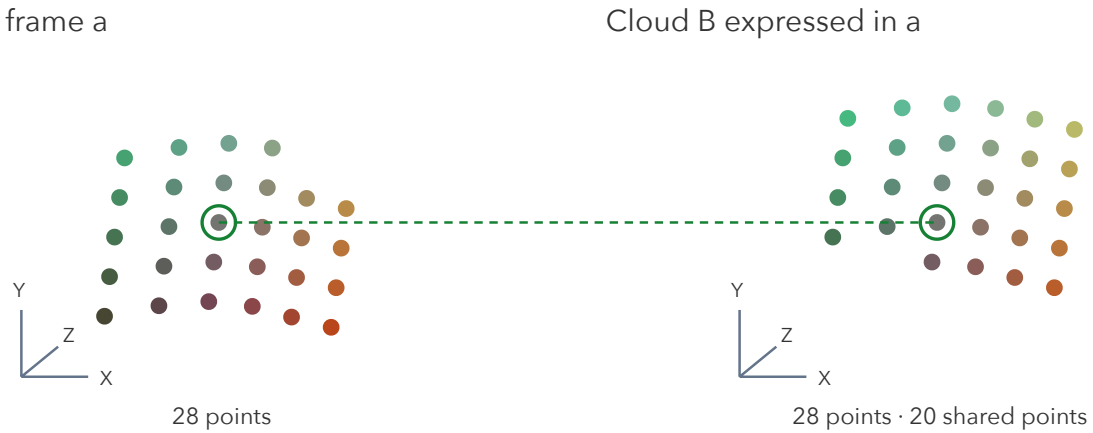
Florence et al., Dense Object Nets, CoRL 2018 · Fig. 2(a), all five configurations · arXiv v2

Finding a grasp point



Florence et al., Dense Object Nets, CoRL 2018 · Fig. 6(i), tail, standard-so · arXiv v2

Extension to 3D correspondence



$$p^a = R_b^a p^b + t_b^a$$

Query p^a = [-0.18, -0.15, 0.49] · matched p^b = [0.54, -0.48, 0.79]

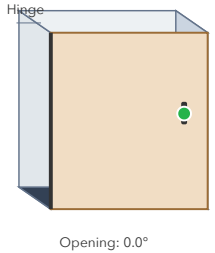
Several reliable, well-distributed correspondences can constrain a rigid registration.

SECTION 07

Learning action possibilities from experience

Observation, action, outcome, target

Earlier observation x_t



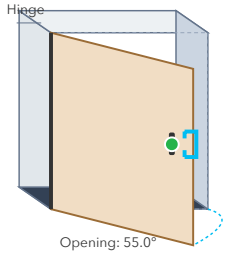
t = 0 · before the action

Commanded action a_t

Grasp handle, pull outward

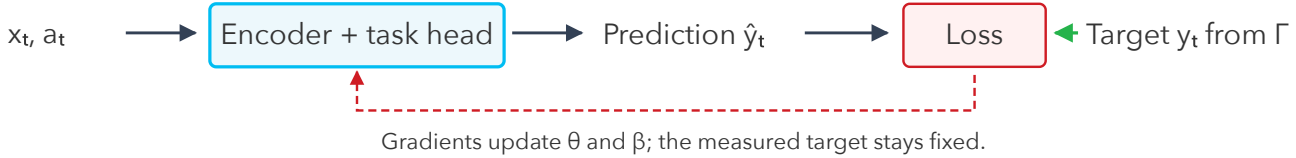
Desired opening: ≥ 30°

Later sensor measurements o



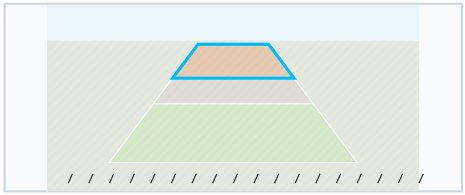
Contact detected · opening 55°

$$y_t = \Gamma(x_t, a_t, o_{t:t+\Delta})$$



Illustrative prediction 0.881 · measured target 1 · binary loss 0.1269

Traversability depends on the robot



Same selected mud patch · same 1.00 m/s command



Narrow wheels / controller A

Measured 0.25 m/s

Loss: 0.75



Broad tracks / controller B

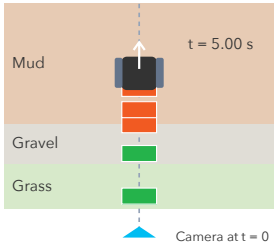
Measured 0.65 m/s

Loss: 0.35

Traversability depends on the platform, controller, action, and outcome rule.

Measuring traversal outcomes

Visited ground · top view



Earlier camera image I_0 · t = 0

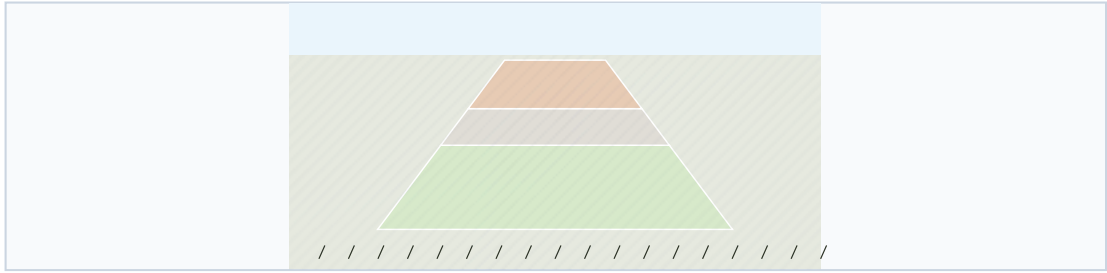


Image recorded → Outcome now available

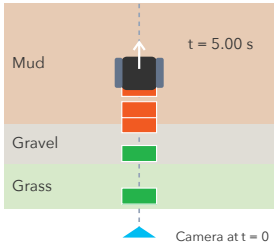
$$y = \Gamma(o) = \mathrm{clip}_{[0,1]}(1 - v_{\mathrm{meas}}/v_{\mathrm{cmd}})$$

Commanded 1.00 m/s · measured 0.60 m/s · loss y = 0.40

This run measures motion; the chosen proxy converts it into a cost.

Delayed supervision

Visited ground · top view



Earlier camera image I_0 · t = 0

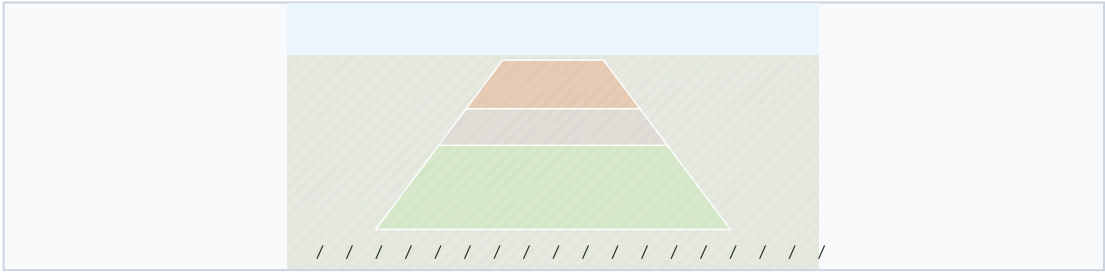


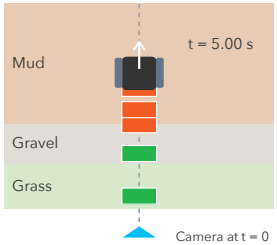
Image recorded → Outcome now available

Commanded 1.00 m/s · measured 0.60 m/s · loss y = 0.40

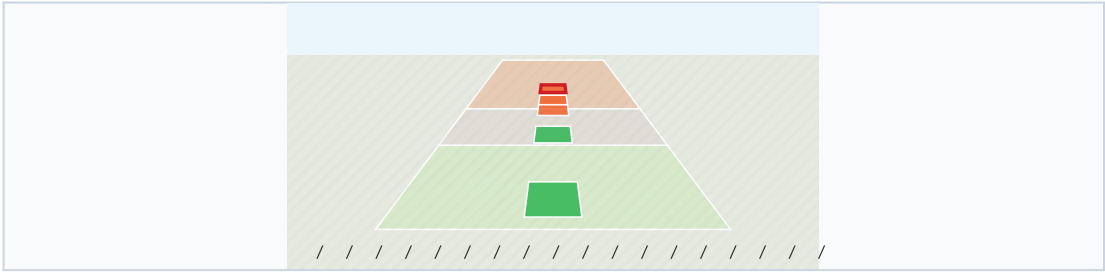
The earlier image is fixed. A target becomes available only after physical traversal.

Reprojecting experience

Visited ground · top view



Earlier camera image I_0 · t = 0



Observed lower cost Observed higher cost Unvisited

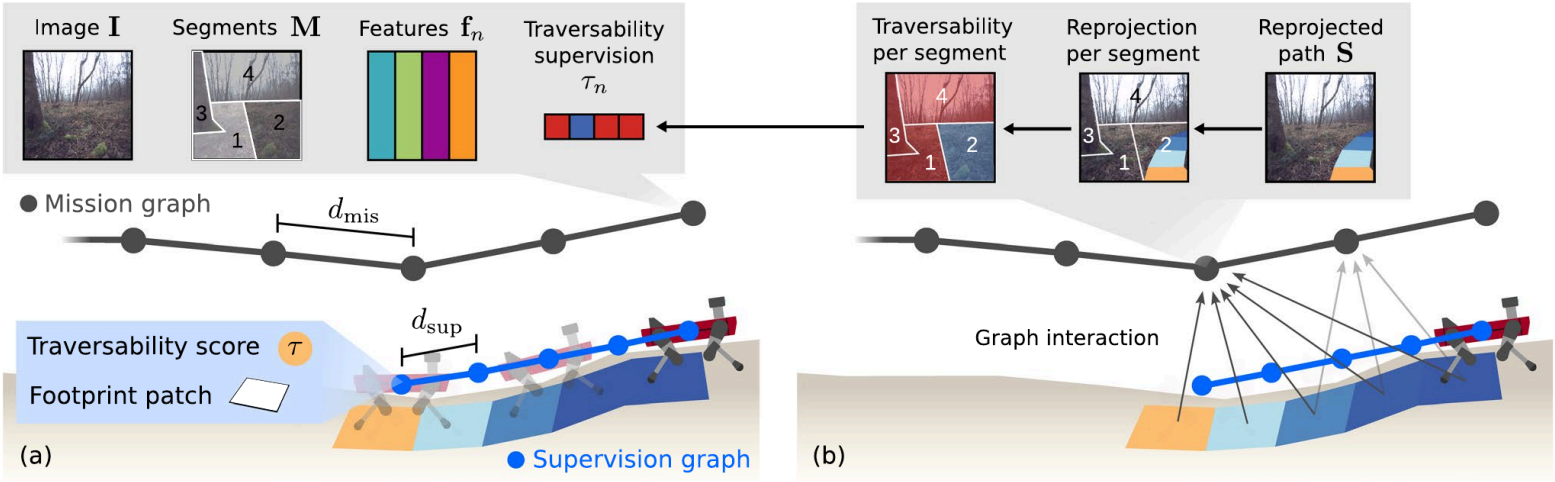
$$\mathbf{u}_0 = \pi\left(K_0\left(R_r^{c_0} p^r + t_r^{c_0}\right)\right)$$

Commanded 1.00 m/s · measured 0.60 m/s · loss y = 0.40 · 5 projected targets · pose time offset 0.0 s

Toy local estimate: 0.500 → measured target 0.40 · mean squared error 0.0690

Restore time alignment before using these measured targets.

Wild Visual Navigation



Visual input

RGB images and pretrained visual features

Measured signal

Commanded versus estimated planar velocity

Accumulated supervision

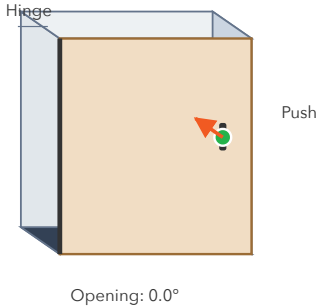
Footprint scores reprojected into stored images

Experienced motion supplies targets for visual traversability estimation.

Mattamala et al., WVN, Autonomous Robots 2025 · Fig. 5(a,b), journal PDF page 6

Manipulation affordances

Fixed initial observation x_t



Pull outward

Initial opening direction

Along the closed door

Little opening moment

Push inward

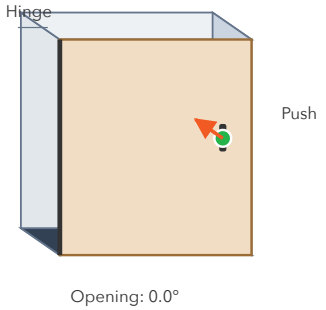
Closed stop blocks motion

Contact radius 0.85 m · direction F = [0.00, 1.00, 0.00] · fixed grasp orientation

The same handle supports different outcomes for different action directions.

Predicting an action's outcome

Fixed initial observation x_t



Pull outward

Initial opening direction

Along the closed door

Little opening moment

Push inward

Closed stop blocks motion

Contact radius 0.85 m · direction F = [0.00, 1.00, 0.00] · fixed grasp orientation

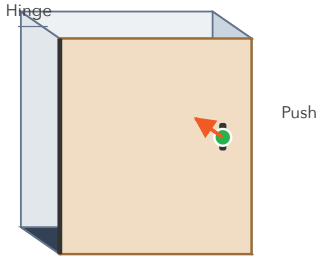
$$\hat{y}_t = h_\beta(f_\theta(x_t), a_t)$$

Predicted opening success: 0.018

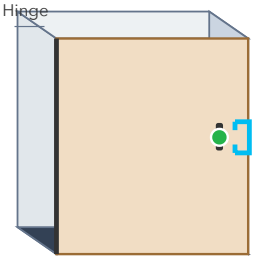
Predict from the observation and proposed action before seeing the outcome.

Automatically constructing an affordance target

Fixed initial observation x_t



Executed observation · complete



Contact radius 0.85 m · direction F = [0.00, 1.00, 0.00] · fixed grasp orientation

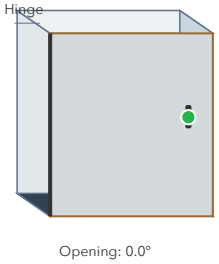
$$y_t = \Gamma(o) = \mathbf{1}[\text{measured opening} \geq 30^\circ]$$

Predicted opening success: 0.018 · Measured opening 0.0° → automatic target y = 0

Task objective: open the door by at least 30°.

ActAIM as an interaction-learning example

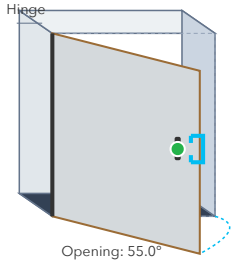
Initial depth D_0



Execute action in simulation

- Contact point p
- Gripper orientation R
- Motion direction F

Later depth D_1



Encode before / after depth → Embedding-change threshold → **Automatic interaction label**

Learn interaction modes
Latent model + local geometry

→

Propose actions
Where and how to interact

The paper learns a distribution of interactions that produce visual change.

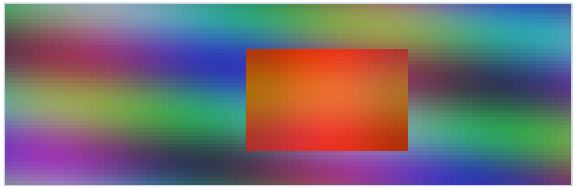
A change criterion does not by itself specify “open this door by 30°”.

Wang et al., ActAIM, ICRA 2023 · Sections III, IV-A/B, V-A · arXiv v1

Summary: the learning principle

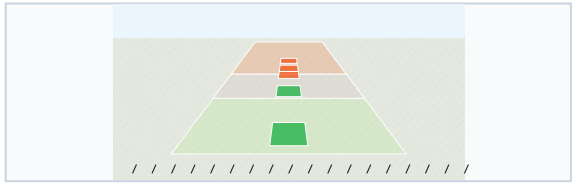
Geometric association

Recorded camera view



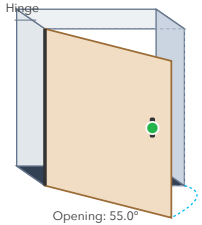
Depth + poses + visibility
→ corresponding pixels

Traversal experience



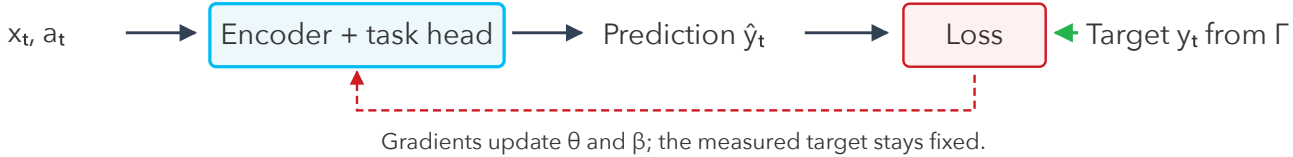
Command + measured motion
→ local cost targets

Cabinet interaction



Measured task outcome
→ success target

$$\hat{y}_t = h_\beta(f_\theta(x_t), a_t), \qquad y_t = \Gamma(x_t, a_t, o_{t:t+\Delta})$$



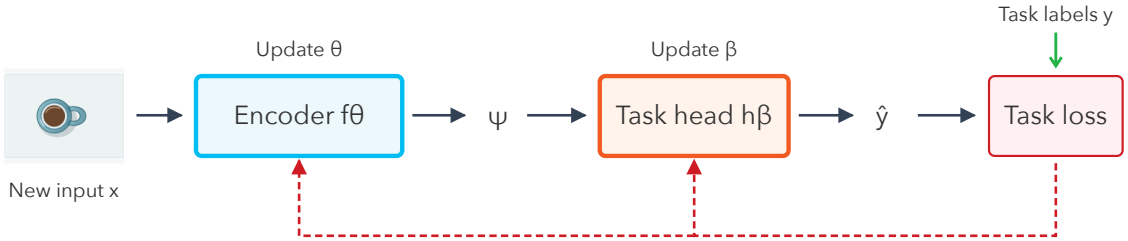
> Identify what supplies the target, which observations it applies to, and what the model must predict.

SECTION 08

Key considerations to remember

Adaptation

Fine-tune encoder + task head



Dark: prediction · green: labels · dashed red: gradients

$$\Theta = (\theta, \beta), \quad \text{update encoder and head}$$

Frozen features

Evaluate information accessible to the chosen head.

Fine-tuning

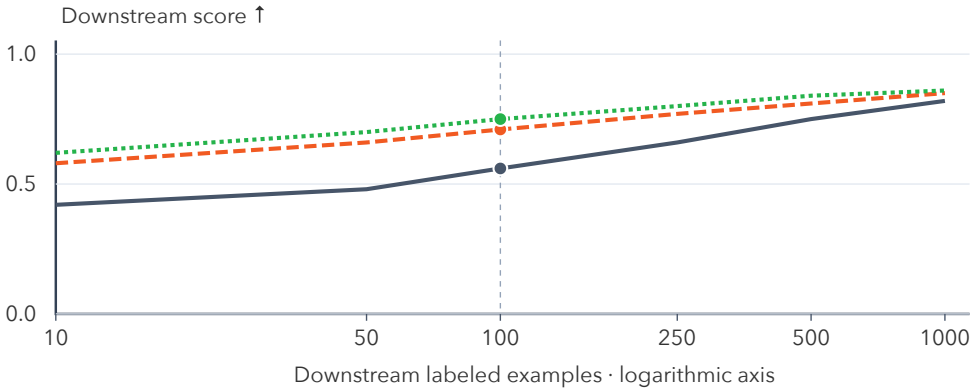
Adapt features and head to the target task.

For a linear probe, hβ is linear. A frozen encoder can also feed a nonlinear head.

Chen et al., SimCLR, ICML 2020 · Appendix B.8

Label efficiency

Illustrative curves



Random initialization **0.56** Supervised pretraining **0.71** Self-supervised pretraining **0.75**

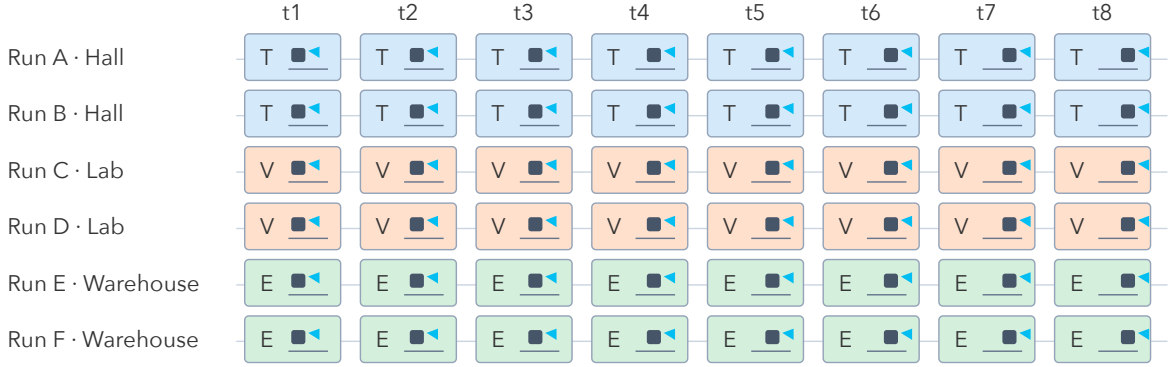
Same downstream split and task head · consistent tuning rules
Report pretraining data, annotations, and compute separately.

Compare label requirements at a stated task score under a declared evaluation protocol.

Splitting robot data correctly

T · Train V · Validation E · Test / evaluation

Time advances →



Train/test audit · adjacent pairs: **0** · shared trajectories: **0** · shared environments: **0**

Whole environments are separated in these metadata; verify the actual recordings too.

Choose the split for the claim: new run, recording session, object, or environment.

scikit-learn · grouped-data evaluation; robot grouping shown here is illustrative

Evaluating the intended perception task

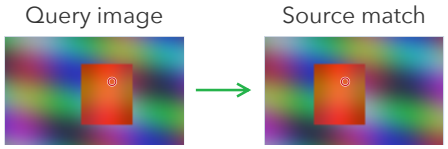
Depth

AbsRel = mean(|D̃ − D| / D)

Reference D	2 m	4 m	6 m
Prediction D̃	2.0 m	4.0 m	6.0 m

AbsRel: **0.000** · median-aligned

Correspondence



Pixel error / match recall on visible held-out pairs

Traversability



Predicted cost error on held-out experienced patches

Manipulation affordance



Pull at handle

Predicted opening

success: **0.881**

Held-out outcome classification or probability loss

Scale alignment uses reference depth at evaluation; it does not recover metric scale at inference.

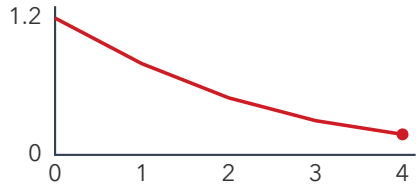
Predictive error and closed-loop robot success answer different questions.

Monodepth2 official evaluator · AbsRel and monocular median scaling

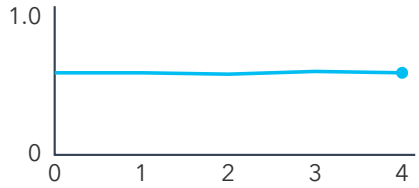
Pretext loss and downstream usefulness

Constructed example · these curves are not an experimental result

Training pretext loss ↓

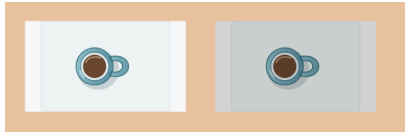


Held-out downstream score ↑



Horizontal axes: optimization checkpoint n

Optimization checkpoint n = 4 · pretext loss 0.18 · downstream score 0.60



→



Background-only features

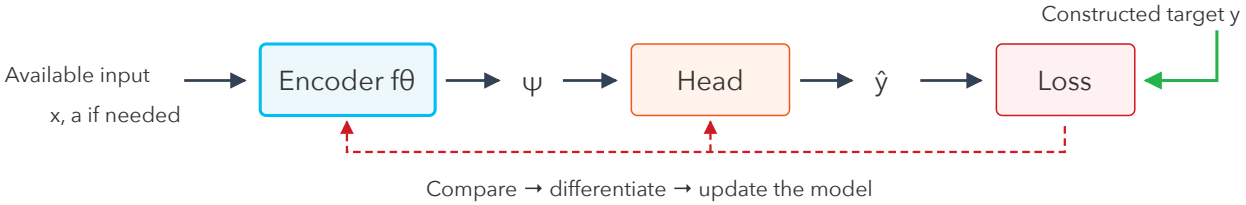
Views agree, but handle position is lost.

A lower pretext loss does not establish improvement on the intended robot task.

Course synthesis

Related views	Hidden observations	Self-Distillation	Geometry	Later measurements
$\tilde{x}_1 \leftrightarrow \tilde{x}_2$	$x_k \rightarrow$ target x_k	$\mathrm{sg}(f_{\bar{\theta}}(x))$	Depth + poses	o → Γ → y
Augmentation / time	Withheld pixels	Target features	Depth / visible matches	Traversal / interaction

Alternative target sources, each with a defined construction rule and supported observations.



What the signal assumes View relations · representation · visibility · timing · measurement support

Takeaway: the same representation can be reused for several tasks.

COURSE SOURCES

References

References: representation learning

SimCLR A Simple Framework for Contrastive Learning of Visual Representations
T. Chen, S. Kornblith, M. Norouzi, G. Hinton. **ICML 2020.**

DINO Emerging Properties in Self-Supervised Vision Transformers
M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, A. Joulin. **ICCV 2021.**

VICReg VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning
A. Bardes, J. Ponce, Y. LeCun. **ICLR 2022.**

MAE Masked Autoencoders Are Scalable Vision Learners
K. He, X. Chen, S. Xie, Y. Li, P. Dollár, R. Girshick. **CVPR 2022.**

I-JEPA Self-Supervised Learning From Images With a Joint-Embedding Predictive Architecture
M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, N. Ballas. **CVPR 2023.**
Official training code: src/train.py

Other sources cited in the course

FixMatch FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence
K. Sohn, D. Berthelot, C.-L. Li, Z. Zhang, N. Carlini, E. D. Cubuk, A. Kurakin, H. Zhang, C. Raffel. **NeurIPS 2020.**

NetVLAD NetVLAD: CNN Architecture for Weakly Supervised Place Recognition
R. Arandjelović, P. Gronat, A. Torii, T. Pajdla, J. Sivic. **CVPR 2016.**

References: geometry and correspondence

Monodepth2

Digging Into Self-Supervised Monocular Depth Estimation

C. Godard, O. Mac Aodha, M. Firman, G. Brostow. **ICCV 2019.**

Official depth evaluator: evaluate_depth.py · commit b017a81

TCN

Time-Contrastive Networks: Self-Supervised Learning from Video

P. Sermanet, C. Lynch, Y. Chebotar, J. Hsu, E. Jang, S. Schaal, S. Levine. **ICRA 2018.**

Dense Object Nets

Dense Object Nets: Learning Dense Visual Object Descriptors By and For Robotic Manipulation

P. R. Florence, L. Manuelli, R. Tedrake. **CoRL 2018.**

References: interaction and evaluation

WVN

Wild visual navigation: fast traversability learning via pre-trained models and online self-supervision

M. Mattamala, J. Frey, P. Libera, N. Chebrolu, G. Martius, C. Cadena, M. Hutter, M. Fallon.
Autonomous Robots 49, article 19, 2025.

ActAIM

Self-Supervised Learning of Action Affordances as Interaction Modes

L. Wang, N. Dvornik, R. Dubeau, M. Mittal, A. Garg. **ICRA 2023.**

Evaluation source

Evaluation

Cross-validation: evaluating estimator performance — grouped-data iterators

scikit-learn developers. **Official documentation · accessed 7 October 2026.**